

ROSOrin Pro 활용 자율주행 로봇 프로그래밍

디자인: 메이커형 · 트랙: 대학교

SV로보틱스

목차

- **00-2** — 전원 인가 및 초기 구동
- **02-1** — ROS2 첫걸음
- **03-1** — 첫 이동 명령
- **03-2** — 바퀴 전방향 테스트
- **06-1** — 라인트레이싱 색상 캘리브레이션
- **07-1** — 물체 색상 추적
- **08-1** — 라이다 장애물 회피
- **09-1** — 로봇팔 첫 동작
- **10-1** — YOLO 물체 인식
- **11-1** — 음성으로 로봇 움직이기
- **12-1** — LLM 기반 자연어 이동 제어
- **13-1** — 쓰레기 분류 로봇
- **14-1** — 색상+AprilTag 물체 분류
- **15-1** — 손 제스처로 로봇 제어하기
- **16-1** — 자율주행 통합
- **17-1** — LLM 함수 호출(function calling)
- **18-1** — LLM 색상 선택 라인트레이싱
- **19-1** — 낙상 감지 로봇
- **20-1** — 사람 따라가기
- **21-1** — 지정한 사람만 따라 움직이기

02. 전원 인가 및 초기 구동

⌚ 0.5시간 (실습 포함)

선수: 01. 로봇 소개 및 사양

시작 전에. 전원 스위치가 **꺼진 상태**에서만 배터리 커넥터를 연결한다.

필요 지식

- 리튬 배터리는 운송 규정상 완충 상태로 출고되지 않으므로 최초 사용 전 충전이 필수다.
- 배터리 극성(적/흑)을 반대로 연결하면 회로 손상 위험이 있다.
- 충전기는 배터리 DC 포트 전용이며, Jetson 메인보드 DC 입력에 연결하면 보드가 손상된다.

단계 별로 따라 하기

1

로봇팔 → 덤스카메라 → (옵션)음성박스 → (옵션)LCD 순서로 조립

판정기준: 나사 8개소 체결 완료

2

전원 OFF 확인 후 배터리 커넥터 연결

판정기준: 적-적/흑-흑 일치

3

충전기를 배터리 DC 포트에 연결, 표시등 확인

판정기준: 표시등 녹색 전환

4

전원 스위치 ON, 약 1분 대기

판정기준: LED1 파란 점멸 + 부저 1회

완료 체크

- ☐ 배터리를 안전하게 연결하고 규정된 순서로 충전할 수 있다.
- ☐ 정상 부팅 여부를 LED·부저(buzzer) 신호로 판별할 수 있다.
- ☐ 부팅 이상 발생 시 SD카드 접촉 불량 등 기본 원인을 진단하고 조치할 수 있다.

더 알아두면 좋은 것

- 리튬 배터리는 과충전·과방전에 민감해서 전용 충전기가 전압을 정밀 제어한다 — 일반 USB 충전기나 다른 전압 어댑터를 쓰면 안 되는 이유다.
- LED·부저(buzzer) 신호 체계는 STM32 컨트롤러가 관리하며, 이 신호를 알아두면 이후 모든 모듈에서 "로봇이 정상인지"를 빠르게 판별할 수 있다.

충전기 연결해도 표시등이 안 켜짐

원인: 콘센트 미연결 또는 충전기 불량

해결: 콘센트 연결 확인 후 다른 콘센트로 재시도

전원 ON 했는데 부저만 울리고 LED는 안 켜짐

원인: 배터리 전압 10V 미만(저전압 경고)

해결: 즉시 충전 후 재시도

SD카드 재장착해도 계속 무반응

원인: SD카드 자체 손상 가능성

해결: 제조사 기술지원 문의

확인 퀴즈

배터리 커넥터를 반대로(역극성으로) 연결하면 어떻게 되는가?

충전 중 표시등 색깔과, 완료됐을 때 표시등 색깔은 각각 무엇인가?

부저가 "삐-삐-삐" 반복해서 울리는 것은 무엇을 의미하는가?

도전 과제. 로봇이 배터리 전압을 ROS2 토픽으로 실시간 발행하는지 `ros2 topic list`로 직접 찾아보고, 있다면 `ros2 topic echo`로 현재 전압을 확인해보세요. 정답을 미리 알려주지 않는 것은 의도한 것입니다 — 스스로 찾는 연습입니다.

02-1. ROS2 첫걸음 — 노드와 토픽

🕒 1시간 (실습 포함)

선수: 00. 로봇 시작하기

시작 전에. 새 터미널을 열 때마다 `source /opt/ros/humble/setup.zsh; source ~/ros2_ws/install/setup.zsh`를 반드시 실행한다.

필요 지식

- ROS2 프로그램은 노드(Node) 단위로 실행되며, 노드는 토픽(Topic)을 통해 메시지를 주고받는다.
- 이 로봇의 ROS2 배포판은 Humble, 작업공간은 `~/ros2_ws`이며 로그인 셸이 `zsh`다.
- 작업환경을 불러오지 않으면 `ros2` 명령 자체가 인식되지 않는다.

📖 선수지식 — 막히면 여기부터

- [ROS2 노드·토픽 개념](#)

토픽 개념 설명 부분 위주 — 글 안에서 저자가 이전 글(노드 개념)도 함께 요약 언급하므로 별도로 안 찾아 봐도 됨

— PinkWink 블로그 (국내 로보틱스 교육자, 한국어)

단계 별로 따라 하기

1

로봇에 SSH 접속 후 작업환경 source

판정기준: `ros2` 명령 정상 인식

2

`ros2 topic list`로 현재 토픽 확인

판정기준: `/controller/cmd_vel` 등 출력

3

`hello_robot.py` 작성 (아래 코드)

판정기준: 코드 저장 완료

4

실행 후 별도 터미널에서 `ros2 node list` 확인

판정기준: 목록에 노드 이름 표시

```
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class HelloRobot(Node):
    def __init__(self):
        super().__init__('hello_robot')
        self.create_timer(1.0, self.tick)
        self.count = 0

    def tick(self):
        self.count += 1
        self.get_logger().info(f'ROS0rin Pro 노드 실행 중... {self.count}')

def main():
    rclpy.init()
    rclpy.spin(HelloRobot())

if __name__ == '__main__':
    main()
```

완료 체크

- ☐ ROS2 작업환경을 활성화하고 현재 실행 중인 노드·토픽 목록을 확인할 수 있다.
- ☐ 최소 구성의 rclpy 노드를 작성하고 실행할 수 있다.
- ☐ 실행 로그와 `ros2 node list` 로 노드의 정상 동작 여부를 확인할 수 있다.

더 알아두면 좋은 것

- 노드(Node)는 프로세스 하나에 대응하고, 토픽(Topic)은 발행-구독(pub-sub) 방식으로 메시지가 흐르는 통로다. 하나의 토픽에 여러 노드가 동시에 구독할 수 있다.
- `ros2 node list`, `ros2 topic echo <토픽>`, `ros2 topic hz <토픽>`는 실행 중인 시스템을 진단하는 기본 CLI 도구다.
- `rclpy.spin()`은 노드를 계속 실행 상태로 유지하며 타이머·구독 콜백을 처리한다 — 이게 없으면 타이머가 한 번도 실행되지 않는다.

막혔다면 (더 보기)

 `ros2: command not found`

원인: 작업환경(source)을 안 함

해결: source 두 줄을 다시 실행

노드를 실행했는데 로그가 전혀 안 뜸

원인: `rclpy.spin()` 호출을 빠뜨림

해결: `main()` 함수에 `spin` 추가

`ros2 node list`에 내 노드가 안 보임

원인: 노드가 이미 종료됐거나 에러로 죽음

해결: 터미널에 남은 에러 메시지 확인

확인 퀴즈

노드와 토픽의 차이를 한 문장으로 설명하시오.

`rclpy.spin()`을 빼먹으면 어떤 문제가 생기는가?

작업환경(source)은 왜 매 터미널마다 새로 실행해야 하는가?

도전 과제. `create_timer`의 주기를 1.0에서 0.5로 바꿔서 로그 출력 빈도가 어떻게 달라지는지 관찰해보세요. 그다음 두 번째 타이머를 추가해서 서로 다른 주기로 다른 메시지를 출력하게 만들어보세요.

03-1. 첫 이동 명령 — cmd_vel로 로봇 움직이기

🕒 1시간 (실습 포함)

선수: 02. ROS2 기초

시작 전에. 실습 전 로봇 주변 1.5m 이상 공간을 확보한다.

필요 지식

- /controller/cmd_vel 토픽에 geometry_msgs/Twist 메시지를 발행하면 로봇이 움직인다.
- linear.x는 전후진(forward/backward) 속도(m/s), angular.z는 회전 속도(rad/s)다.
- 새로 실행한 퍼블리셔에서 --once(1회성) 발행은 구독자와 매칭되기 전에 유실될 수 있음이 실측으로 확인됐다(정지·그리퍼(gripper) 명령 모두 재현). 반드시 반복 발행 방식을 쓴다.

단계 별로 따라 하기

1

CLI로 반복 발행 체험: `ros2 topic pub -r 10 /controller/cmd_vel ...`

판정기준: 로봇이 전진함

2

Ctrl+C 후 별도로 정지 명령(0,0) 발행

판정기준: 로봇이 완전히 정지함

3

first_move.py 작성 (아래 코드) — 5초 전진 후 정지

판정기준: 코드 저장 완료

4

실행 후 관찰

판정기준: 5초간 전진 후 정지

```
#!/usr/bin/env python3
import time, rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class FirstMove(Node):
    def __init__(self):
        super().__init__('first_move')
        self.pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)

    def move_forward(self, seconds=5.0, speed=0.2):
        twist = Twist(); twist.linear.x = speed
        end = time.time() + seconds
        while time.time() < end:
            self.pub.publish(twist)    # 반복 발행 - --once 유실 방지
            time.sleep(0.1)
        self.pub.publish(Twist())      # 정지도 발행으로

def main():
    rclpy.init()
    FirstMove().move_forward()
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

완료 체크

- ☐ Twist 메시지의 구조(linear.x, angular.z)를 이해하고 로봇을 전진(forward)·회전(rotation)시킬 수 있다.
- ☐ 1회성 명령 발행이 유실될 수 있는 이유를 알고, 안전한 반복 발행(repeated publish) 방식을 적용할 수 있다.

더 알아두면 좋은 것

- Twist 메시지는 linear(x,y,z)·angular(x,y,z) 총 6개 값을 갖지만, 지상 바퀴로봇은 보통 linear.x·linear.y·angular.z 3개만 쓴다 — z축 이동이나 x/y축 회전은 물리적으로 불가능하기 때문이다.
- 퍼블리셔의 큐 크기(create_publisher 세 번째 인자)는 메시지가 밀릴 때 몇 개까지 버퍼링할지 정한다. 이동 제어처럼 "최신 값만 중요한" 경우 작게(1) 유지하는 게 일반적이다.

 --once로 보냈는데 로봇이 안 움직임

원인: 발행이 구독자와 매칭되기 전에 유실(실측 확인된 버그)

해결: `-r 10`으로 반복 발행

 코드 실행 후 로봇이 계속 움직임(안 멈춤)

원인: 정지 코드가 실행되지 않고 프로그램이 강제 종료됨

해결: `try/finally`로 정지 코드 실행을 보장

 속도값을 크게 줬는데 이상하게 움직임

원인: 권장 범위($-0.6 \sim 0.6$ m/s) 초과

해결: 범위 내 값으로 조정

확인 퀴즈

--once가 유실될 수 있는 이유는 무엇인가?

linear.x가 음수면 로봇은 어떻게 움직이는가?

정지 명령도 반복 발행해야 하는 이유는 무엇인가?

도전 과제. `move_forward` 함수를 확장해서 "사각형 경로로 주행하기"(전진 → 회전(rotation) 90도 → 이 과정을 4회 반복)를 구현해보세요.

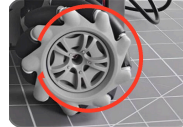
03-2. 바퀴 전방향 테스트 — 전진(forward)·평행이동(lateral translation)·회전(rotation) 모두 확인하기

⌚ 1시간 (실습 + 결과기록 포함)

선수: 03-1. 첫 이동 명령



전진(forward)
회전



후진
(backward) 회
전

바퀴 회전 방향 ↔ 로봇 진행 방향 — 네 바퀴 모두 사진과 같은 방향으로 돌면 (왼쪽) 로봇은 앞으로, 반대 방향으로 돌면(오른쪽) 뒤로 갑니다. 이 관계는 명확합니다. 반면 평행이동(lateral translation)이나 제자리 회전(rotation)은 바퀴마다 서로 다른 방향으로 돌아야 나오는 조합 동작이라, 어떤 바퀴가 어느 방향으로 돌아야 어느 쪽으로 가는지는 로봇마다·문서마다 표기가 갈릴 수 있어 여기서 단정하지 않았습니다 — 오늘 실습에서 `linear.y`·`angular.z`를 직접 넣어보고 실제 방향을 관찰·기록하는 것이 바로 이 부분을 확인하는 과정입니다.

시작 전에. 실습 전 로봇 전후좌우(대각선 포함) 1.5m 이상 공간을 확보한다.

필요 지식

- ROSOrin Pro는 메카넘 휠 구동이라 일반 바퀴와 달리 옆으로 평행이동도 가능하다 — 바퀴 롤러가 45도로 붙어있어 네 바퀴 조합으로 이런 움직임이 나온다. (공식 매뉴얼 2.2절)
- `/controller/cmd_vel` 토픽(geometry_msgs/Twist)의 `linear.x`(전후진)·`linear.y`(좌우 평행이동)·`angular.z`(회전)를 조합해 발행한다.
- 확인된 값: `linear.x` 권장 범위는 $-0.6 \sim 0.6$ m/s이다. `angular.z`는 양수면 좌회전(반시계), 음수면 우회전(시계방향)이다. (공식 매뉴얼 2.1.5절)
- `linear.y`의 부호가 실제로 좌/우 중 어느 쪽인지는 공식 문서에 명시돼 있지 않다 — 이번 실습에서 직접 테스트하고 기록하는 것 자체가 학습 목표다.

- 03-1에서 배운 반복 발행(repeated publish) 원칙(--once 유실 방지)이 여기에도 동일하게 적용된다.

선수지식 — 막히면 여기부터

- [메카닉 휠 역기구학\(수식 유도\)](#)

페이지 중반 'Inverse kinematics' 섹션만 보면 됨 — $v_x \cdot v_y \cdot \omega$ 를 4개 바퀴 속도로 바꾸는 행렬식과 대각선 이동 계산 예시가 나옴. 앞부분(휠 구조 설명)은 건너뛰어도 무방
— Ecam Eurobot (로봇 경진대회 팀의 오픈 튜토리얼, 영문)

단계별로 따라하기

1

전진(linear.x=+0.2) → 후진(linear.x=-0.2) 테스트

판정기준: 앞/뒤로 직진 이동

2

linear.y=+0.2로 평행이동 테스트 → 실제 방향(좌/우) 관찰

판정기준: 옆으로 이동, 방향 기록

3

linear.y=-0.2로 반대 방향 평행이동 테스트

판정기준: 반대쪽으로 이동 확인

4

angular.z=+0.3(좌회전(turn left, CCW)) → -0.3(우회전) 테스트

판정기준: 제자리에서 회전

5

linear.x+linear.y 동시 입력으로 대각선 이동 테스트

판정기준: 대각선 방향으로 이동

6

결과 기록표 작성

판정기준: 6개 이동 유형 모두 기록

```

#!/usr/bin/env python3
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist

class WheelTest(Node):
    def __init__(self):
        super().__init__('wheel_test')
        self.pub = self.create_publisher(Twist, '/controller/cmd_vel', 1)

    def move(self, vx=0.0, vy=0.0, wz=0.0, seconds=2.0):
        twist = Twist()
        twist.linear.x = vx
        twist.linear.y = vy
        twist.angular.z = wz
        end = time.time() + seconds
        while time.time() < end:
            self.pub.publish(twist)    # 반복 발행 - --once 유실 방지
            time.sleep(0.1)
        self.pub.publish(Twist())      # 정지도 발행으로

def main():
    rclpy.init()
    node = WheelTest()
    node.move(vx=0.2, seconds=2.0)      # 1. 전진
    time.sleep(1.0)
    node.move(vy=0.2, seconds=2.0)      # 2. 평행이동(방향은 관찰해서 기록)
    time.sleep(1.0)
    node.move(wz=0.3, seconds=2.0)      # 4. 회전
    rclpy.shutdown()

if __name__ == '__main__':
    main()

```

결과 기록

시도	이동 유형	입력값 (VX, VY, WZ)	실제 관찰된 방향	판정
1	전진	0.2, 0, 0		성공/실패
2	후진	-0.2, 0, 0		성공/실패
3	평행이동 (+y)	0, 0.2, 0	좌/우 중 실제로 관찰된 방향	성공/실패
4	평행이동 (-y)	0, -0.2, 0		성공/실패
5	좌/우회전	0, 0, ± 0.3		성공/실패
6	대각선 이동			성공/실패

한 줄 소감

linear.y의 실제 부호-방향 대응, 대각선 이동 시 속도 감소 여부 등을 자유롭게 기록

완료 체크

- ☐ Twist 메시지의 linear.x·linear.y·angular.z를 조합해 로봇을 여러 방향으로 움직일 수 있다.
- ☐ 각 축의 부호(+/-)가 실제로 어느 방향에 대응하는지 직접 실험으로 확인하고 기록할 수 있다.
- ☐ 전진·후진·좌우 평행이동·좌우회전 6가지 이동을 모두 시도하고 결과를 기록으로 남길 수 있다.

더 알아두면 좋은 것

- 메카넘 휠(mecanum wheel) 4개는 각각 다른 방향의 대각선 롤러를 가진다 — 네 바퀴를 같은 방향으로 돌리면 전진, 대각선 방향으로 짝을 지어 다르게 돌리면 평행이동(lateral translation)이 나온다. "네 바퀴 회전 속도의 조합"이 곧 로봇의 최종 이동 방향을 결정한다.
- 대각선 이동은 vx와 vy를 동시에 넣는 것이라 각 바퀴에 걸리는 부하가 순수 전진·평행이동보다 커서, 실제 속도가 이론값보다 느려질 수 있다.

막혔다면 (더보기)

평행이동 시 로봇이 살짝 회전(rotation)하며 이동함

원인: 바퀴 캘리브레이션 편차 또는 무게중심 치우침

해결: 공식 매뉴얼 1.7절 속도 캘리브레이션 절차 참고

🔧 대각선 이동이 잘 안 됨

원인: $v_x \cdot v_y$ 값이 너무 작아 마찰을 못 이김

해결: 두 값을 각각 조금씩 키워서 재시도

🔧 제자리 회전이 아니라 이동하면서 회전함

원인: `angular.z`만 넣었는데 이전 `linear` 값이 남아있음

해결: 매번 새 `Twist()` 객체를 생성해서 사용

확인 퀴즈

메카닉 휠이 일반 바퀴와 다른 점은 무엇인가?

대각선 이동은 어떤 값들의 조합으로 만들어지는가?

왜 대각선 이동을 가장 마지막, 가장 넓은 공간에서 시도하라고 했는가?

도전 과제. $v_x \cdot v_y \cdot w_z$ 세 값을 동시에 조합해서 "회전하면서 전진"하는 나선형 경로를 만들어보세요.

🎓 메카닉 휠 역기구학(inverse kinematics)

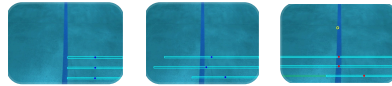
일반적인 메카닉 휠 로봇에서, 로봇이 원하는 속도(v_x : 전진, v_y : 평행이동, ω : 회전)를 네 바퀴의 개별 회전속도로 변환하는 공식은 흔히 다음과 같은 형태를 갖는다: $v_1 = v_x - v_y - \omega(l_x + l_y)$, $v_2 = v_x + v_y + \omega(l_x + l_y)$, $v_3 = v_x + v_y - \omega(l_x + l_y)$, $v_4 = v_x - v_y + \omega(l_x + l_y)$ ($l_x \cdot l_y$ 는 바퀴 중심과 로봇 중심 사이의 가로·세로 거리). 이 로봇의 정확한 하위 컨트롤러 구현(펌웨어)이 이 공식을 그대로 쓰는지 이번 세션에서 직접 확인하지 못했다 — ROS2 계층에서는 `/controller/cmd_vel`에 $v_x \cdot v_y \cdot \omega$ 만 넘기면 그 이후 변환은 하위 컨트롤러가 담당한다.

이 공식이 왜 필요한가 — 일반 바퀴(디퍼렌셜 드라이브)는 v_y (옆으로 이동)를 만들 방법이 아예 없다. 메카닉 휠의 롤러가 45도로 붙어있어서 네 바퀴 회전속도의 조합만으로 v_y 성분을 만들어낼 수 있는 것이다. 이번 실습에서 `linear.y` 부호를 직접 실험으로 확인한 것이 바로 이 역기구학 결과를 몸으로 검증한 것이다.

06-1. 라인트레이싱 색상 캘리브레이션 — 왜 지정한 색을 못 찾았을까?

🕒 1시간 (실측 디버깅 실습)

선수: 03-2. 바퀴 전방향 테스트



- ① 픽셀좌표로 시도 — 실패 ② 정규화 좌표로 재 시도 — 성공 ③ 3-스캔 라인 가중치 분석

실측으로 잡은 진짜 버그 —

set_target_color 서비스에 픽셀좌표 (예: 310,320)를 그대로 보냈더니①, 소스코드 안에서 `x = int(point.x * 이미지폭)`로 다시 곱해져 좌표가 화면 밖으로 튕겨나갔습니다. 화면 비율(0~1)로 바꿔 보내자② 정확히 테이프 위에 점이 찍혔습니다. 여기에 더해 로봇과 가장 가까운 스캔라인이 전체 조향 판단의 70%를 차지한다는 것도③ 코드에서 직접 확인했습니다 — 코드를 읽지 않았다면 절대 몰랐을 원인입니다.

시작 전에. 이번 레슨은 set_running을 켜지 않고(정지 상태) 캘리브레이션만 진행한다 — 로봇이 움직이지 않는다.

필요 지식

- line_following 앱은 enter(진입) → set_target_color(기준색 지정) → set_running(주행 시작/정지) → exit(종료) 순서의 서비스 호출로 제어한다.
- 확인된 사실** — set_target_color가 받는 x, y는 픽셀이 아니라 **0~1 사이 정규화 좌표(normalized coordinate)**다. 실제 소스코드(ColorPicker.__call__)에서 `x = int(point.x * 이미지폭)`으로 폭을 곱해 픽셀로 환산하는 것을 확인했다.
- 3개 스캔라인(scan line)의 가중치는 로봇에 가까운 순서대로 **0.7 · 0.2 · 0.1**이다 — 가장 가까운 라인 하나가 전체 조향 판단의 70%를 차지한다.
- LAB 색공간은 밝기(L)와 색상(A·B)을 분리해서 표현해 조명 변화에 상대적으로 강하다 — RGB보다 색 추적에 자주 쓰인다.

선수지식 — 막히면 여기부터

- LAB 색공간 정의(RGB와 다른 점)

$L^*(\text{밝기}) \cdot a^*(\text{빨강-초록}) \cdot b^*(\text{파랑-노랑})$ 3채널 정의 부분만 보면 됨 — MATLAB 실습 코드 부분은 건너뛰어도 무방

— MathWorks 공식 도움말(한국어)

단계별로 따라하기

1

예측: 픽셀좌표(310, 320)를 그대로 보내면 어떻게 될지 먼저 예상해서 적는다

판정기준: 예측 기록 완료

2

enter → set_target_color(픽셀값) → 결과 화면 캡처

판정기준: 사진①과 비교

3

결과가 예측과 다르면, app/common.py의 ColorPicker 소스코드를 읽고 원인을 찾는다

판정기준: 버그 지점 특정

4

좌표를 0~1 비율로 변환해서 set_target_color 재호출 → 결과 캡처

판정기준: 사진②와 비교

5

결과 기록표 작성 및 소견 서술

판정기준: 표 작성 완료

```
ros2 service call /line_following/enter std_srvs/srv/Trigger {}
```

(실패) 픽셀좌표를 그대로 보냄 — 화면 밖으로 튕겨나감

```
ros2 service call /line_following/set_target_color \
  interfaces/srv/SetPoint "{data: {x: 310.0, y: 320.0, z: 0.0}}"
```

(성공) 0~1 사이 정규화좌표로 변환해서 보냄

```
ros2 service call /line_following/set_target_color \
  interfaces/srv/SetPoint "{data: {x: 0.484, y: 0.8, z: 0.0}}"
```

```
ros2 service call /line_following/set_running std_srvs/srv/SetBool "{data: true}"
```

결과 기록

시도	보낸 좌표	예측 (찾을까?)	실제 관찰 결과	판정
1	x=310, y=320 (픽셀값 그대로)	찾는다/못찾는다		성공/실패
2	x=0.484, y=0.8 (정규화)			성공/실패
3	본인이 고른 다른 좌표			성공/실패

한 줄 소감

왜 픽셀좌표는 실패하고 정규화좌표는 성공했는지, 자기 말로 설명해서 적기

완료 체크

- ☐ 색상 기반(color-based) 물체 인식이 LAB 색공간 임계값(threshold)으로 동작하는 원리를 설명할 수 있다.
- ☐ ROS2 서비스 3종(Trigger·SetBool·커스텀 SetPoint)으로 비전 앱을 켜고 끄고 설정할 수 있다.
- ☐ 소스코드를 직접 읽어 좌표계(픽셀 vs 정규화) 불일치 버그의 원인을 스스로 진단할 수 있다.
- ☐ 스캔라인 가중치 구조에서 로봇과 가까운 영역이 왜 더 중요한지 설명할 수 있다.

더 알아두면 좋은 것

- 스캔라인 가중치가 $0.7 \cdot 0.2 \cdot 0.1$ 로 로봇에 가까울수록 높은 이유 — 제어이론 관점에서 "지금 당장 눈 앞의 상황"이 "저 멀리의 상황"보다 즉각적인 조향 반응에 더 중요하기 때문이다. 이런 구조를 프리뷰 제어(preview control)라 부르기도 한다.
- 같은 이유로, 근접 스캔라인 하나가 잘못 검출되면 먼 두 라인이 맞아도 전체 판단이 크게 틀어진다 — 실측으로 확인한 이번 사례가 정확히 그 경우다.

막혔다면 (더 보기)

좌표를 넣었는데 항상 화면 구석 근처의 색을 잡음

원인: 픽셀좌표를 정규화좌표 자리에 넣어 값이 클램프(clamp)됨

해결: 좌표를 이미지폭·높이로 나눠 0~1 사이 값으로 변환

정규화좌표로도 여전히 못 찾음

원인: 카메라 화이트밸런스로 인한 전체적인 색조 편향(예: 청색 캐스트)

해결: 편향이 덜한 지점에서 재샘플링하거나 threshold 값 조정

스캔라인 3개 중 일부만 라인을 못 찾을

원인: 라인이 화면의 그 영역(특히 로봇 근접부)까지 안 이어짐

해결: 물리적으로 라인·로봇 위치를 재정렬

확인 퀴즈

set_target_color가 받는 좌표는 픽셀인가 비율인가?

3개 스캔라인 중 가중치가 가장 높은 것은 어디에 있는 라인인가?

근접 스캔라인 하나가 오검출되면 전체 조향에 어떤 영향을 주는가?

도전 과제. 지금 코드의 weight_sum은 항상 1.0으로 고정돼 있다. 실제로 검출에 성공한 스캔라인의 가중치만 더해서 weight_sum을 동적으로 계산하도록 고쳐보고, 근접 라인이 하나 빠졌을 때 결과가 어떻게 달라지는지 비교해보세요.

07-1. 물체 색상 추적 — 카메라로 공을 쫓아가는 로봇 만들기

🕒 1시간 (실습 + 결과기록 포함)

선수: 06-1. 라인트레이싱 색상 캘리브레이션

시작 전에. 이번에도 처음엔 `set_running`을 켜지 않고 좌표·색상 인식만 먼저 확인한다.

필요 지식

- `object_tracking.py`는 `line_following`과 같은 4개 서비스 (`enter.set_target_color.set_running.exit`)를 그대로 사용한다 — 06-1에서 확인한 "좌표는 0~1 정규화값" 원칙이 여기서도 동일하게 적용된다. (실제 소스:
`ObjectTrackingNode.set_target_color_srv_callback → ColorPicker(request.data, 10)`)
- 추적은 PID 제어기 2개로 이뤄진다 — `pid_yaw`(좌우 회전, $K_p=0.006$)는 물체 x좌표가 화면 중앙 (`x_stop=320px`)에서 벗어난 만큼 회전시키고, `pid_dist`(전후 이동, $K_p=0.002$)는 물체 y좌표가 목표 정지 위치(`y_stop`)에서 벗어난 만큼 전진·후진시킨다. (소스: `ObjectTracker.__call__`)
- `y_stop`은 처음 물체를 찾은 순간의 y좌표로 자동 고정된다 — 즉 "처음 발견했을 때의 거리"가 기준 정지거리가 된다. (소스: `if not self.y_stop_record: ... self.y_stop = int(y)`)
- 로봇 조향 방식이 Ackermann(`MACHINE_TYPE`에 'Acker' 포함)이면 회전을 조향각·회전반경(R) 계산으로 변환하고, 메카넘 계열이면 `angular.z`를 직접 사용한다 — 같은 "회전하라"는 판단도 하드웨어 구조에 따라 실제 변환 코드가 달라진다. (소스: `ObjectTracker.__call__`의 'Acker' in `self.machine_type` 분기)

단계 별로 따라 하기

1

enter 호출 → 카메라 스트림에서 물체 위치 확인

판정기준: 스트림 정상 표시

2

set_target_color에 정규화 좌표로 물체 색 지정 → get_target_color로 인식된 RGB 확인

판정기준: RGB 값 응답 확인

3

set_running=false 상태에서 물체를 좌우로 움직여 결과 화면에 원이 따라오는지만 관찰(로봇은 정지 상태)

판정기준: 원이 물체를 따라 이동

4

set_running=true로 전환 → 실제 추적 주행 관찰

판정기준: 로봇이 물체 방향으로 회전

5

물체를 가까이/멀리 움직여 pid_dist가 전후 이동을 만드는지 확인

판정기준: 전진/후진 반응 확인

6

결과 기록표 작성

판정기준: 4가지 테스트 모두 기록

```

ros2 service call /object_tracking/enter std_srvs/srv/Trigger {}

# 0~1 정규화좌표로 목표색 지정 (06-1과 동일 원칙)
ros2 service call /object_tracking/set_target_color \
  interfaces/srv/SetPoint "{data: {x: 0.5, y: 0.4, z: 0.0}}"

ros2 service call /object_tracking/get_target_color std_srvs/srv/Trigger {}

# 좌표·색상 인식만 확인 (로봇은 정지 상태)
ros2 service call /object_tracking/set_running std_srvs/srv/SetBool "{data: false}"

# 실제 추적 주행 시작
ros2 service call /object_tracking/set_running std_srvs/srv/SetBool "{data: true}"

```

결과 기록

시 도	테스트 내용	예상 반응	실제 관찰	판정
1	물체를 좌우로 이동	로봇이 물체 쪽으로 회전		성공/실패
2	물체를 가까이 이동			성공/실패
3	물체를 멀리 이동			성공/실패
4	물체를 화면 밖으로 치움(놓침)	로봇이 어떻게 반응할지		성공/실패

한 줄 소감

물체를 놓쳤을 때 로봇이 실제로 어떻게 반응했는지, 왜 그렇게 반응했다고 생각하는지 소스코드 근거와 함께 서술

완료 체크

- ☐ object_tracking 앱의 서비스 구조(enter·set_target_color·set_running·exit)가 06-1의 line_following과 동일한 패턴임을 확인할 수 있다.
- ☐ PID 제어기 2개(pid_yaw·pid_dist)가 각각 좌우 회전과 전후 이동을 어떻게 담당하는지 설명할 수 있다.
- ☐ set_target_color에도 0~1 정규화 좌표가 쓰인다는 06-1의 결론이 이 앱에도 그대로 적용됨을 실습으로 확인할 수 있다.
- ☐ 물체와의 정지 거리(y_stop)가 어떻게 정해지는지 소스코드로 확인하고, 실제 추적 동작과 연결지어 설명할 수 있다.

더 알아두면 좋은 것

- PID 세 항(P·I·D) 중 여기서는 P(비례)항만 쓰인다(Kp만 0이 아니고 I·D는 0으로 설정됨) — 목표에서 벗어난 만큼 비례해서만 반응하는 가장 단순한 제어 방식이다.
- lost_target_count 변수는 선언되고 0으로 리셋만 될 뿐, 실제로 증가시키는 코드는 소스 전체에 없다 — "물체를 몇 번 놓쳤는지 세서 재탐색하는" 로직은 이름만 있고 아직 구현되지 않은 상태임을 소스코드에서 직접 확인할 수 있다. 대신 물체를 놓치면(circle이 None) 기본값 Twist()(전부 0)가 그대로 반환되어 로봇이 자연스럽게 멈춘다.

막혔다면 (더 보기)

추적을 시작해도 로봇이 가만히 있음

원인: set_running이 여전히 false이거나 color_picker가 아직 클리어되지 않음

해결: get_target_color로 색이 실제로 지정됐는지 먼저 확인

물체가 가까운데도 계속 다가감

원인: y_stop이 처음 발견 시점의 값으로 고정되어, 그 이후 목표 정지거리가 바뀌지 않음

해결: set_target_color를 다시 호출해 y_stop을 재설정

회전이 너무 급격하거나 느림

원인: pid_yaw의 Kp(0.006)가 로봇·환경에 맞지 않음

해결: 심화 과제처럼 Kp 값을 조정해 재시도

확인 퀴즈

object_tracking과 line_following이 공유하는 서비스 4개는 무엇인가?

pid_yaw와 pid_dist는 각각 어떤 축(방향)을 담당하는가?

y_stop 값은 언제, 어떻게 정해지는가?

도전 과제. pid_yaw·pid_dist의 Kp 값(0.006, 0.002)을 바꿔가며 로봇의 반응 속도가 어떻게 달라지는지 관찰해보세요. 값을 너무 크게 하면 어떤 현상(진동·오버슈트)이 생기는지도 함께 확인해보세요.

08-1. 라이다 장애물 회피 — 가까운 쪽 반대로 피하기

🕒 1시간 (실습 + 결과기록 포함)

선수: 03-2. 바퀴 전방향 테스트

시작 전에. 실습 전 로봇 주변(특히 전방) 2m 이상 공간을 확보한다 — 회피 중 회전 반경이 커질 수 있다.

필요 지식

- 이 로봇의 라이다(Coin-D6 TOF)는 전원 인가 직후 거리값이 0 또는 NaN으로 나오는 현상이 실측으로 확인되어 있고, 공식 절차대로 12분 이상 워밍업하면 정상화된다(정확한 원인은 규명되지 않음) — 실습 전 `ros2 topic echo /scan_raw`로 값이 정상인지 먼저 확인한다. (출처: 이전 세션 로봇 실기 테스트 인수인계 기록)
- `set_running` 서비스는 정수 하나(0~3)를 받는다 — 0 종료, 1 장애물 회피, 2 라이다 추종, 3 라이다 경비. 이번 레슨은 모드 1만 다룬다. (소스: `LidarController.set_running_srv_callback` 주석)
- 모드 1의 판단 로직은 전방 240도(`MAX_SCAN_ANGLE`) 스캔 데이터를 좌·우로 나눠 각각의 최단거리를 구하고, 왼쪽이 가까우면 오른쪽으로 · 오른쪽이 가까우면 왼쪽으로 · 양쪽 다 가까우면 180도 회전한다. (소스: `LidarController.lidar_callback`의 `running_mode == 1` 분기)
- `threshold`(기본 0.6m)와 `scan_angle`(스캔 각도)은 `set_param` 서비스로 실시간 조정할 수 있다 — `threshold`는 0.3~1.5m, `scan_angle`은 0~90도 범위를 벗어나면 서비스가 실패 응답을 반환하도록 소스코드에 범위 검증이 들어있다. (소스: `set_parameters_srv_callback`)

📖 선수지식 — 막히면 여기부터

• [PID 제어 기초\(P·I·D 각 항의 역할과 수식\)](#)

글 상단 수식과 'P/PID' 3단 비교 부분만 보면 충분 — 뒤쪽 Ziegler-Nichols 튜닝법·C 의사코드는 심화 참고용이라 안 봐도 됨

— Sinduy's blog (CC BY 4.0, 한국어)

단계별로 따라하기

1

ros2 topic echo /scan_raw로 거리값이 0/NaN이 아닌지 확인

판정기준: 정상적인 거리값(m) 출력

2

enter 호출

판정기준: 서비스 응답 success

3

set_running(1)로 장애물 회피 모드 시작

판정기준: 로봇이 직진 시작

4

로봇 왼쪽에 장애물 배치 → 반응 관찰, 이어서 오른쪽 → 양쪽 다 막고 반응 관찰

판정기준: 장애물 반대 방향으로 회전

5

set_param으로 threshold를 낮춰(예: 0.4) 재시도 → 반응 거리 변화 관찰

판정기준: 더 가까이에서 반응

6

결과 기록표 작성

판정기준: 4가지 케이스 모두 기록

```
# 실습 전 라이다 워밍업 확인 (값이 0이거나 NaN이면 대기)
ros2 topic echo /scan_raw --once

ros2 service call /lidar_app/enter std_srvs/srv/Trigger {}

# 모드 1 = 장애물 회피
ros2 service call /lidar_app/set_running interfaces/srv/SetInt64 "{data: 1}"

# threshold(m), scan_angle(deg), speed(m/s) 순서
ros2 service call /lidar_app/set_param interfaces/srv/SetFloat64List \
  "{data: [0.4, 90.0, 0.2]}"
```

결과 기록

시 도	장애물 위치	예상 반응	실제 반응	판정
1	왼쪽만	오른쪽으로 회전 예상		성공/실패
2	오른쪽만			성공/실패
3	양쪽 다	180도 회전 예상		성공/실패
4	threshold 0.4로 낮춘 후 왼쪽만			성공/실패

한 줄 소감

threshold를 낮췄을 때 로봇이 장애물에 얼마나 더 가까이 다가간 후에 반응했는지, 체감한 차이를 자유롭게 기록

완료 체크

- ☐ 라이다 스캔 데이터를 좌·우 절반으로 나눠 각각의 최단거리를 비교하는 장애물 회피 로직의 원리를 설명할 수 있다.
- ☐ `set_running(0~3)` 4가지 모드 중 이번 레슨이 다루는 모드 1(장애물 회피)을 실제로 실행하고 관찰할 수 있다.
- ☐ 라이다는 전원 인가 직후 워밍업이 필요하다는 이전 실측 사실을 알고, 정상 데이터가 나오는지 먼저 확인한 뒤 실습을 시작할 수 있다.
- ☐ `threshold`(회피 시작 거리)를 바꿔가며 로봇의 반응 시점이 어떻게 달라지는지 비교할 수 있다.

더 알아두면 좋은 것

- 이 코드는 로봇 조향 방식(메카넘 vs Ackermann, `MACHINE_TYPE`에 'Acker' 포함 여부)에 따라 회피 로직 자체가 완전히 다른 두 갈래로 나뉜다 — Ackermann 차량은 조향각 특성상 단순 제자리 회전이 불가능해서, 회전반경(R)을 계산해 부드럽게 회피하도록 별도로 구현되어 있다. (소스: `lidar_callback`의 `if 'Acker' not in self.machine_type: ... else: ...` 두 분기)
- 모드 2(라이다 추종)와 모드 3(라이다 경비)는 PID 제어기(`pid_yaw`·`pid_dist`)로 "가장 가까운 물체와의 각도·거리"를 유지하는 방식이라, 모드 1(단순 좌우 비교)보다 정교하지만 그만큼 구조도 복잡하다 — 이번 레슨에서는 다루지 않는다.

막혔다면 (더 보기)

장애물이 있는데 반응이 없음

원인: 라이다 워밍업 미완료로 거리값이 0/NaN

해결: `/scan_raw` 값을 확인하고 12분 이상 대기 후 재시도

가까이 가도 반응이 늦음

원인: `threshold`가 너무 작게 설정됨

해결: `set_param`으로 `threshold`를 높여 재시도(1.5m 이하 범위)

회피하다가 다시 같은 장애물 쪽으로 돌

원인: 직전 회피 방향(`last_act`)이 번갈아 반전되도록 설계되어 있어 좁은 공간에서 진동할 수 있음

해결: 장애물 사이 폭을 넓히거나 `threshold`를 낮춰 재시도

확인 퀴즈

`set_running`에 넣는 정수 1은 어떤 모드를 의미하는가?

라이다 실습 전에 왜 `/scan_raw` 값을 먼저 확인해야 하는가?

왼쪽과 오른쪽 모두 장애물이 있을 때 로봇은 어떻게 반응하는가?

도전 과제. MAX_SCAN_ANGLE(240도)을 좁혀서(예: 120도) 로봇이 더 정면의 장애물에만 반응하도록 바꿔 보고, 넓은 공간에서 옆을 스쳐 지나가는 물체에 불필요하게 반응하는지 비교해보세요.

🎓 PID 제어 이론과 이 코드의 게인(gain) 값

PID 제어기는 목표값과 현재값의 차이(오차, error)에 세 가지 방식으로 반응한다 — P(비례, Proportional)는 오차 크기에 비례해서, I(적분, Integral)는 오차가 누적된 양에 비례해서, D(미분, Derivative)는 오차가 변하는 속도에 비례해서 반응한다. 최종 출력은 $output = K_p \cdot error + K_i \cdot \int error \, dt + K_d \cdot (d \, error / dt)$ 형태다.

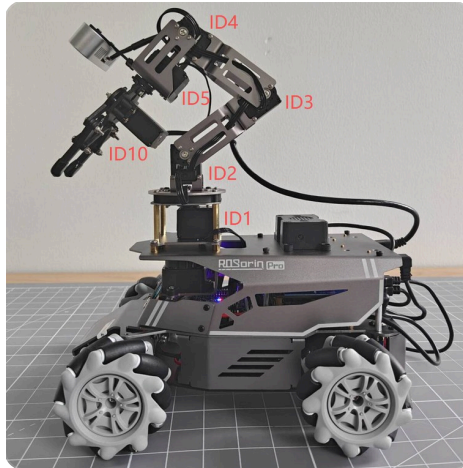
이 lidar_controller.py의 pid_yaw는 PID(1.6, 0, 0.16), pid_dist는 PID(1.7, 0, 0.16)로 설정되어 있다 — I(적분)항은 0으로 꺼져 있고 P와 D만 쓰는 'PD 제어기'다. I항을 안 쓰는 이유로 흔히 드는 것은, 라이다 추종처럼 목표가 계속 움직이는 상황에서 I항이 오차를 계속 누적하면 과거의 오차까지 끌고 가서 오히려 진동·불안정을 키우기 쉽기 때문이다(다만 이 코드에 그런 설계 의도가 주석으로 명시되어 있지는 않다 — 일반적인 제어이론 관점의 해석이다).

D(미분)항이 0이 아니라는 것은, 오차가 빠르게 변할 때(장애물이 갑자기 가까워질 때 등) 그 변화에 미리 반응해서 진동(overshoot)을 억제하려는 의도로 흔히 해석된다.

09-1. 로봇팔 첫 동작 — 서보 (servo) 각도로 관절(joint) 움직이기

🕒 1.5시간 (실습 + 결과기록 포함)

선수: 03. 이동 제어



서보 ID ↔ 관절 대응(공식 매뉴얼 8장 기준) —
ID1 베이스 회전(팬) · ID2 어깨(shoulder) ·
ID3 팔꿈치(elbow) · ID4·ID5 손목(wrist) ·
ID10 그리퍼(gripper). 오늘은 이 중 **ID3(팔꿈치)**를 움직여봅니다.

시작 전에. 로봇팔이 움직이는 동안 관절 사이에 손가락을 넣지 않는다.

필요 지식

- 로봇팔은 지능형 버스 서보 6개로 구성된다 — ID1 베이스 회전(팬), ID2·3·4 팔 관절, ID5 손목, ID10 그리퍼. (공식 매뉴얼 8장)
- 서보 위치값은 0~1000 범위이며, 중앙값(약 500)이 정자세에 해당한다. 실제 각도가 아닌 이 정수값으로 목표를 지정한다.
- 서보 명령은 /servo_controller 토픽(ServosPosition 메시지)으로 발행하며, 공식 헬퍼 함수 `set_servo_position(pub, duration, ((id, position), ...))` (`servo_controller.bus_servo_control`)를 사용하는 것이 권장 방식이다.

1

위 사진에서 ID3(팔꿈치)의 실제 위치를 눈으로 확인

판정기준: 움직일 관절을 정확히 가리킬 수 있음

2

아래 코드 작성 — ID3을 중앙값(500)에서 소량만 이동

판정기준: 코드 저장 완료

3

실행 전 예측 기록 → 실행 → 실제로 움직인 관절이 ID3(팔꿈치)가 맞는지 관찰

판정기준: 지정한 관절만 움직였는지 확인

4

예측과 실제를 비교해 실습 결과 기록표 작성

판정기준: 기록표 4번 항목 작성 완료

```
#!/usr/bin/env python3
import time
import rclpy
from rclpy.node import Node
from servo_controller_msgs.msg import ServosPosition
from servo_controller.bus_servo_control import set_servo_position

class ArmFirstMove(Node):
    def __init__(self):
        super().__init__('arm_first_move')
        self.joints_pub = self.create_publisher(ServosPosition, '/servo_controller',
1)

        time.sleep(0.5) # 퍼블리셔 연결 대기

    def move_joint(self, servo_id, position, duration=1.5):
        # position: 0~1000 (중앙 약 500), duration: 초 단위
        set_servo_position(self.joints_pub, duration, ((servo_id, position),))
        self.get_logger().info(f'서보 {servo_id} -> {position} ({duration}s)')

def main():
    rclpy.init()
    node = ArmFirstMove()
    node.move_joint(3, 520, 1.5) # ID3(팔꿈치)를 중앙에서 살짝만 이동
    time.sleep(2.0)
    rclpy.shutdown()

if __name__ == '__main__':
    main()
```

결과 기록

시도	서보 ID	목표값	지속시간(S)	관찰 결과	판정
1					성공/부분/실패
2					성공/부분/실패
3					성공/부분/실패

한 줄 소감

ID3(팔꿈치)을 중앙값 500에서 520으로 움직이면 실제로 어느 방향으로 얼마나 움직일지 예상해서, 실행하기 전에 먼저 적기

완료 체크

- ☐ 사진에 표시된 서보 ID(1, 2, 3, 4, 5, 10)와 실제 관절 부위를 정확히 대응시켜 설명할 수 있다.
- ☐ 서보 ID·목표값(0~1000)·지속시간을 지정해 로봇팔 관절을 안전하게 움직일 수 있다.
- ☐ 실행 전 예측과 실제 결과를 비교하고, 그 차이의 이유를 논리적으로 서술할 수 있다.

더 알아두면 좋은 것

- 버스 서보(servo)는 하나의 시리얼 라인에 여러 서보가 데이지체인으로 연결되고, 각 서보의 ID로 개별 주소 지정이 가능한 방식이다 — 여러 조명을 하나의 리모컨으로 각각 제어하는 것과 비슷한 원리다.
- set_servo_position()은 여러 서보를 동시에 움직이는 것도 지원한다 — 튜플을 여러 개 넣으면(예: ((2, 500), (3, 520))) 여러 관절(joint)이 동시에 목표 위치로 이동한다.

막혔다면 (더 보기)

🔧 코드를 실행했는데 서보가 전혀 안 움직임

원인: 서보 컨트롤 노드가 안 떠 있거나 다른 프로세스가 시리얼 포트를 점유 중

해결: `ros2 node list`로 서보 컨트롤러 노드 확인

🔧 서보가 목표와 다른 위치로 감

원인: 서보 자체 편차(deviation)가 누적된 상태

해결: 제조사 PC 소프트웨어로 서보 편차 보정 (공식 매뉴얼 8.1.2절)

🔧 그리퍼(gripper)가 완전히 안 닫히거나 이상한 소리가 남

원인: 기계적 한계에 도달, 서보 손상 위험

해결: 즉시 반대 방향으로 조정

확인 퀴즈

position 값 500은 대략 몇 도(240도 범위 기준)에 해당하는가?

서보 ID를 여러 개 튜플로 넣으면 어떤 일이 일어나는가?

그리퍼(ID10)를 처음 테스트할 때 왜 반드시 빈손이어야 하는가?

도전 과제. ID2·ID3·ID5 세 관절을 순서대로 움직이는 `move_joint` 호출을 여러 번 배치해서, "손 흔들기"처럼 보이는 동작 시퀀스를 만들어보세요.

10-1. YOLO 물체 인식 — 모델 이름 하나가 클래스 목록과 검출 방식을 함께 정한다

🕒 1시간 (실습 + 결과기록 포함)

선수: 07-1. 물체 색상 추적

시작 전에. 이번 레슨은 로봇이 움직이지 않는다 — 인식 결과만 확인한다.

필요 지식

- yolo_node.py는 /yolo/start·/yolo/stop 단 2개의 Trigger 서비스만 제공한다 — line_following-object_tracking의 enter/set_target_color/set_running/exit 4종 패턴과 다르다. (소스: YoloNode.__init__의 create_service 2줄)
- launch 파일(yolo_detect.launch.py)에서 model_name에 'garbage'가 들어있으면 12개 쓰레기 분류 클래스 + obb(회전된 박스) 방식, 'traffic'이 들어있으면 6개 신호·표지 클래스 + detect(일반 박스) 방식, 그 외엔 COCO 80개 클래스 + detect 방식이 자동으로 선택된다 — 모델 이름 문자열 하나로 클래스 목록과 검출 방식이 함께 정해진다. (소스: yolo_detect.launch.py의 model_name 분기)
- conf(신뢰도 임계값) 기본값은 0.6이다 — YOLO가 이 값 이상 확신하는 검출 결과만 ObjectsInfo로 발행한다. (소스: launch 파일의 conf 기본값)
- ~/yolo_start_detect 서비스는 프로그램 시작과 동시에 만들어지지 않는다 — /yolo/start가 호출되고 카메라 프레임이 최소 1장 처리된 뒤에야 image_proc 루프 안에서 생성된다. "서비스 목록에 아직 안 보인다"가 곧 오류는 아니라는 것을 소스코드로 확인할 수 있다. (소스: image_proc 안의 if not self.yolo_detet_flag: self.create_service(...))

선수지식 — 막히면 여기부터

- [신뢰도\(confidence\) 임계값과 precision/recall 트레이드오프](#)

confidence threshold를 올리고 내릴 때 precision·recall이 어떻게 반대로 움직이는지 설명하는 부분 위주 — YOLO 전용 글은 아니지만 개념은 이 레슨의 conf 파라미터에 그대로 적용됨
— velog 기술 블로그 (한국어)

1

yolo_detect.launch.py를 기본값(model_name=yolo26n)으로 실행 →
/yolo/start 호출

판정기준: 서비스 응답 success

2

카메라에 **COCO** 클래스 물체(병·컵·휴대폰 등)를 비취 결과 화면에 박스가 그려지는
지 확인

판정기준: 박스·클래스명 표시

3

ros2 topic echo ~/object_detect로 class_name·score 값 직접 확인

판정기준: 메시지 정상 수신

4

conf 값을 낮춰(예: 0.3) 재실행 → 오탐지(엉뚱한 물체 검출)가 느는지 관찰

판정기준: 오탐지 빈도 비교

5

model_name을 **traffic**으로 바꿔 재실행 → 클래스 목록이 6개로 바뀌는지 확인

판정기준: go/right/park/red/green/crosswalk만 표시

6

결과 기록표 작성

판정기준: 3가지 설정 모두 기록

```
ros2 launch example yolo_detect.launch.py conf:=0.6 model_name:=yolo26n

ros2 service call /yolo/start std_srvs/srv/Trigger {}

ros2 topic echo /yolo/object_detect

# conf를 낮춰 재실행 (오탐지 비교용)
ros2 launch example yolo_detect.launch.py conf:=0.3 model_name:=yolo26n

# 클래스 목록·검출 방식이 함께 바뀌는지 확인
ros2 launch example yolo_detect.launch.py model_name:=traffic
```

결과 기록

시도	설정 (MODEL_NAME / CONF)	인식 시도 물체	실제 결과 (CLASS_NAME · SCORE)	판정
1	yolo26n / 0.6			성공/실패
2	yolo26n / 0.3	1번과 동일한 물체		성공/실패
3	traffic / 0.6		클래스 목록이 6개로 줄었는지	성공/실패

한 줄 소감

conf를 낮췄을 때 오탐지가 실제로 늘었는지, model_name에 따라 클래스 목록·검출 방식이 어떻게 달라졌는지 소스코드 근거와 함께 서술

완료 체크

- ☐ yolo 앱이 /yolo/start:/yolo/stop 단 2개의 서비스만으로 제어된다는 것을 line_following·object_tracking의 4개 서비스 패턴과 비교해 설명할 수 있다.
- ☐ launch 파일의 model_name 파라미터 하나가 클래스 이름 목록과 검출 방식(detect/obb)을 동시에 결정하는 구조를 이해하고, 실제로 다른 model_name으로 실행해 차이를 관찰할 수 있다.
- ☐ conf(신뢰도 임계값) 파라미터를 바꿔가며 오탐지·미탐지가 어떻게 달라지는지 실습으로 확인할 수 있다.
- ☐ ObjectsInfo 메시지에 담기는 정보(class_name·score·box·angle)를 직접 echo해서 확인할 수 있다.

더 알아두면 좋은 것

- OBB(oriented bounding box, 회전된 박스)와 일반 박스(detect)는 저장하는 좌표 형식부터 다르다 — obb는 중심좌표·너비·높이·회전각(xywhr)을, detect는 좌상단·우하단 좌표(xyxy)를 사용한다. 쓰레기 분류처럼 물체가 기울어져 놓일 수 있는 경우에만 obb를 쓰는 이유가 여기 있다. (소스: yolo_node.py의 is_obb_task 분기)
- class_name은 classes 리스트에서 class_id로 찾아오는데, 모델이 classes 리스트보다 많은 클래스로 학습되어 있었다면 "ID:숫자" 형태로 표시된다(코드의 fallback) — launch 파일의 클래스 이름 목록이 실제 모델과 정확히 일치해야 사람이 읽을 수 있는 이름이 나온다.

막혔다면 (더 보기)

박스는 그려지는데 이름이 ID:3 같이 나옴

원인: launch의 classes 리스트가 실제 모델 학습 클래스 수·순서와 안 맞음

해결: model_name에 맞는 올바른 클래스 리스트로 재실행

🔧 인식이 하나도 안 됨

원인: conf 임계값이 너무 높거나 물체가 클래스 목록에 없음

해결: conf를 낮춰보고, COCO 80개 클래스에 있는 물체로 재시도

🔧 ~/yolo_start_detect 서비스가 안 보임

원인: /yolo/start를 아직 호출하지 않았거나 카메라 프레임이 한 장도 처리되지 않음

해결: /yolo/start 호출 후 카메라 앞에 물체를 비춰 최소 1프레임 처리시키기

확인 퀴즈

yolo 앱은 몇 개의 서비스로 제어되는가? line_following과 몇 개 다른가?

model_name에 'garbage'가 들어가면 어떤 두 가지가 함께 바뀌는가?

class_name이 "ID:7"처럼 숫자로만 나온다면 무엇을 의심해봐야 하는가?

도전 과제. FPS 계산 코드($\text{self.fps} = \text{self.fps} \cdot 0.9 + \text{current_fps} \cdot 0.1$)는 지수이동평균(EMA) 방식입니다. 계수를 0.9에서 0.5로 바꾸면 화면에 표시되는 FPS 값이 더 빨리/느리게 반응하는지 실습으로 확인해 보고, 그 이유를 설명해보세요.

🎓 지수이동평균(EMA)과 FPS 계산

$\text{self.fps} = \text{self.fps} \cdot 0.9 + \text{current_fps} \cdot 0.1$ 은 지수이동평균(Exponential Moving Average, EMA)의 전형적인 형태다 — 일반화하면 $\text{EMA}_t = \alpha \cdot \text{EMA}_{(t-1)} + (1-\alpha) \cdot x_t$ 이고, 여기서 $\alpha=0.9$ 다. 매 프레임 새로 측정된 순간 FPS(x_t)는 화면에 표시되는 값에 10%만 반영되고, 나머지 90%는 이전까지 누적된 값을 그대로 이어받는다.

이게 왜 필요한가 — 순간 FPS는 프레임마다 크게 흔들린다(카메라 읽기·추론 시간이 매번 조금씩 다르므로). α 를 1에 가깝게 두면(예: 0.9) 화면에 찍히는 숫자가 부드럽게 변하지만, 실제 성능이 갑자기 나빠져도 반영되기까지 시간이 걸린다(반응 지연, lag). 반대로 α 를 낮추면(예: 0.5) 실제 변화에 빠르게 반응하지만 프레임마다 숫자가 들쭉날쭉해 보인다 — 이게 advancedTask에서 0.9와 0.5를 비교해보라고 한 이유다.

이 구조는 신호처리의 1차 저역통과 필터(first-order low-pass filter)와 수학적으로 동일하다 — 여기서는 FPS 표시에만 쓰였지만, 센서 노이즈를 부드럽게 하고 싶은 어떤 값(예: 배터리 전압, 거리 측정값)에도 같은 공식을 그대로 적용할 수 있다. α 값 하나가 "얼마나 부드럽게, 얼마나 빠르게 반응할지"라는 트레이드오프를 전부 결정한다는 것이 이 공식의 핵심이다.

11-1. 음성으로 로봇 움직이기 — 웨이크워드부터 '이리와'까지

🕒 1시간 (실습 + 결과기록 포함)

선수: 08-1. 라이다 장애물 회피

시작 전에. 음성 명령 중에도 로봇은 실제로 움직인다 — 사방 1m 이상 공간을 확보한다.

필요 지식

- 이 앱의 웨이크워드는 `awake_node.py`의 파라미터 기본값 기준 "小幻小幻"(샤오환샤오환)이다 — 같은 프로젝트의 `voice_control_move.py` 로그 메시지는 영어로 "hello hiwonder", 한글 설명으로는 "샤오환샤오환"이라고 서로 다르게 적어놓아서 혼동을 일으킨다. 실제 동작을 정하는 것은 `awake_node.py`의 파라미터이지 이 로그 문자열이 아니다. (소스: `voice_control_move.py` 67~69행 vs `awake_node.py` 197행)
- `/asr_node/voice_words` 토픽으로 인식된 한글 단어("전진"·"후진"·"좌회전"·"우회전"·"이리와")를 받아 각각 다른 Twist 값을 만들고, 2초 뒤(`time_stamp`) 자동으로 정지시킨다 — 한 번 말하면 2초간 움직이고 스스로 멈추는 방식이다. (소스: `VoiceControlMoveNode.main`)
- "이리와"는 다른 명령과 달리 2단계로 동작한다 — 먼저 `/awake_node/angle`(마이크 배열이 감지한 화자 방향)만큼 회전(`angular.z=±1.0`)하고, 회전이 끝나면 `lidar_follow` 플래그가 켜져 라이다 추종 모드(`start_follow`)로 전환된다. (소스: `main()`의 '이리와' 분기, 236~238행)
- "이리와" 명령은 `machine_type`이 'ROSOrin_Acker'이면 조건문에서 아예 제외된다(`elif ... and self.machine_type != 'ROSOrin_Acker'`) — Ackermann 조향 로봇은 이 기능 자체가 없다. (소스: `main()`의 `elif` 조건문)

단계별로 따라하기

1

웨이크워드("샤오환샤오환") 발화 → 웨이크업 성공 신호(효과음) 확인

판정기준: 효과음 재생

2

15초 이내에 "전진" 발화 → 로봇이 2초간 전진 후 자동 정지하는지 관찰

판정기준: 2초 이동 후 정지

3

같은 방식으로 "후진"·"좌회전"·"우회전" 각각 테스트

판정기준: 각 명령대로 동작

4

로봇에서 1~2m 떨어져 "이리와" 발화 → 로봇이 화자 방향으로 회전한 뒤 다가오는 지 관찰

판정기준: 회전 후 접근

5

결과 기록표 작성

판정기준: 5가지 명령 모두 기록

동작 확인용 토픽 구독 (음성 명령은 서비스 호출이 아니라 발화로 트리거됨)

```
ros2 topic echo /asr_node/voice_words
```

```
ros2 topic echo /awake_node/angle
```

결과 기록

시 도	발 화 명 령	예 상 동 작	실 제 동 작	판 정
1	웨이크워드(샤오환 샤오환)	효과음 재생		성공/실패
2	전진			성공/실패
3	좌회전			성공/실패
4	이리와	회전 후 접근		성공/실패

한 줄 소감

완료 체크

- ☐ 벤더 소스코드에서 "웨이크워드가 무엇인가"를 확인할 때는 로그 문자열이 아니라 실제 동작을 결정하는 파라미터 기본값을 봐야 한다는 것을 실제 사례로 이해한다.
- ☐ 기본 음성 명령 4종(전진·후진·좌회전·우회전)이 각각 어떤 Twist 값과 지속시간으로 변환되는지 설명할 수 있다.
- ☐ "이리와" 명령이 라이다 각도 정보로 화자 방향을 향해 회전한 뒤 라이다 추종 모드로 전환되는 2단계 동작임을 설명할 수 있다.
- ☐ Ackermann 조향 로봇에서는 "이리와" 명령 자체가 지원되지 않는다는 것을 소스코드 조건문에서 확인할 수 있다.

더 알아두면 좋은 것

- "이리와" 명령의 회전각 계산은 감지된 각도가 90도보다 크고 270도보다 작은지에 따라 회전 방향·잔여 회전량 계산식이 달라진다 — 마이크 배열이 보고하는 각도 기준(0~360도, 로봇 정면 기준)을 이해해야 이 분기가 왜 이렇게 나뉘는지 알 수 있다.
- count 변수가 10에 도달하면(연속 10회 정지 상태) start_follow를 자동으로 끈다 — "가까이 다 왔다고 판단되면 추종을 스스로 멈추는" 타임아웃 성격의 안전장치다.

막혔다면 (더 보기)

웨이크업이 안 됨

원인: 웨이크워드 발음이 정확하지 않거나 마이크 하드웨어 문제

해결: 정확한 발음(샤오환샤오환)으로 재시도, 마이크 연결 상태 확인

전진 등은 되는데 이리와만 안 됨

원인: MACHINE_TYPE이 ROSOrin_Acker인 경우 이 기능 자체가 코드에서 제외됨

해결: 로봇 사양 확인(메카넘 계열인지)

이리와 회전은 하는데 그 후 다가오지 않음

원인: 라이다 데이터가 비정상(0/NaN)이라 추종 로직이 작동하지 않음

해결: 08-1의 라이다 워밍업 절차 먼저 수행

확인 퀴즈

이 프로젝트 소스코드에서 로그의 "hello hiwonder"와 실제 파라미터 값 "샤오환샤오환"이 다른 이유는 무엇이라고 추정되는가?

음성 명령 하나를 말하면 로봇은 얼마나 오래 움직이는가?

"이리와" 명령은 왜 Ackermann 조향 로봇에서 지원되지 않는가?

도전 과제. time_stamp 기반 "2초 후 자동 정지" 로직을 실제 코드에서 찾아, 이 값을 1초로 바꾸면 체감상 로봇 반응이 어떻게 달라지는지 실습해보세요.

12-1. LLM 기반 자연어 이동 제어 — 문장 한 줄이 좌표·시간 값으로 바뀌는 과정

🕒 1시간 (실습 + 결과기록 포함)

선수: 11-1. 음성으로 로봇 움직이기

시작 전에. LLM이 예상 밖의 action 값을 생성할 수 있으므로 사방 1.5m 이상 공간을 확보한다.

필요 지식

- 시스템 프롬프트(PROMPT)는 LLM에게 "x·y 선속도(m/s)·z 각속도(rad/s)·t 지속시간(s)"의 4개 값 배열을 순서대로 담은 JSON을 출력하도록 지시한다 — 실제 이동은 이 JSON을 그대로 Twist 메시지로 변환해 발행하는 방식이다. (소스: `process()`의 `msg.linear.x=i[0]`, `msg.linear.y=i[1]`, `msg.angular.z=i[2]`, `time.sleep(i[3])`)
- ASR_LANGUAGE 환경변수는 'Chinese'와 그 외(영어) 두 갈래만 구분하는 조건문(`if os.environ["ASR_LANGUAGE"] == 'Chinese'`)으로 프롬프트를 고른다 — 이 프로젝트의 한글화 작업으로 'Chinese' 분기 프롬프트의 텍스트 내용 자체는 한국어로 번역돼 있지만, 분기 조건 문자열('Chinese')과 else 분기(영어 프롬프트)는 바뀌지 않았다. 즉 실제 배포 환경의 ASR_LANGUAGE 값이 정확히 무엇인지 직접 확인해야 어느 프롬프트가 실제로 쓰이는지 알 수 있다. (소스: `llm_control_move.py` 22번째 줄 분기)
- action 리스트의 마지막 항목은 항상 `[0.0, 0.0, 0.0, 0.0]`이어야 한다고 프롬프트에 명시되어 있다 — 로봇이 명령 실행 후 반드시 멈추도록 LLM에게 요청하는 안전장치다. 다만 이는 프롬프트 "요청"일 뿐 코드가 별도로 강제 검증하지는 않는다. (소스: PROMPT 요구사항 2번, `process()`에 마지막 값 강제 확인 코드 없음)
- 웨이크업(재호출)이 감지되면 interrupt 플래그가 켜지고, 다음 action 실행 반복에서 즉시 Twist() (정지)를 발행하고 나머지 동작을 건너뛴다 — 실행 중인 이동을 즉시 취소할 수 있는 구조다. (소스: `wakeup_callback`, `process()` 안의 `if self.interrupt: ... break`)

선수지식 — 막히면 여기부터

- [eval\(\)의 보안 위험과 안전한 대안](#)

'위험' 섹션과 '안전한 사용법·대안' 섹션 위주로 — `eval()` 기본 사용법 설명(도입부)은 이미 코드로 봤으니 건너뛰어도 됨

— Leapcell 기술 블로그 (한국어)

1

웨이크워드 발화 → LLM 노드가 응답 대기 상태인지 확인

판정기준: 웨이크업 신호 확인

2

"앞으로 2초 이동한 다음 왼쪽으로 1초 회전해줘"처럼 프롬프트 예시와 비슷한 문장으로 명령

판정기준: 지시대로 이동

3

로봇의 실제 동작과 음성 응답(response) 문구를 함께 기록

판정기준: 응답·동작 모두 기록

4

동작 실행 중 다시 웨이크워드를 불러 interrupt가 실제로 동작을 멈추는지 확인

판정기준: 즉시 정지

5

"공중부양해줘"처럼 존재하지 않는 동작을 명령해 action이 빈 리스트로 나오고 응답만 오는지 확인

판정기준: 동작 없이 응답만

6

결과 기록표 작성

판정기준: 4가지 케이스 모두 기록

```
# 프롬프트 요구사항 핵심 발췌 (llm_control_move.py)
# "action": [[x1,y1,z1,t1], [x2,y2,z2,t2], ..., [0.0,0.0,0.0,0.0]]
# x,y ∈ [-1.0, 1.0] (선속도, m/s) / z ∈ [-1.0, 1.0] (각속도, rad/s) / t (지속시간, s)

# action -> Twist 변환 (process() 내부, 실제 소스)
for i in action_list:
    msg = Twist()
    msg.linear.x = float(i[0])
    msg.linear.y = float(i[1])
    msg.angular.z = float(i[2])
    self.mecanum_pub.publish(msg)
    time.sleep(i[3])
    if self.interrupt:
        self.interrupt = False
        self.mecanum_pub.publish(Twist())
        break
```

결과 기록

시도	발화 명령	예상 동작 (ACTION 개수)	실제 동작 · 응답	판정
1	단일 동작 (예: 앞으로 2초 이동)	1개 + 정지		성공/실패
2	복합 동작 (전진 + 회전)			성공/실패
3	동작 중 웨이크워드로 interrupt	즉시 정지		성공/실패
4	존재하지 않는 동작(공중 부양)	action=[]		성공/실패

한 줄 소감

ASR_LANGUAGE 실제 값 확인 결과와, 그에 따라 어느 언어의 프롬프트가 실제로 쓰이고 있었는지 소스코드 근거와 함께 서술

완료 체크

- ☐ 사용자의 자연어 명령이 시스템 프롬프트를 거쳐 [x, y, z, t] 형식의 이동 명령 리스트로 변환되는 전체 흐름을 설명할 수 있다.
- ☐ ASR_LANGUAGE 환경변수가 'Chinese'와 그 외(영어) 두 값만 구분한다는 것, 그리고 이 프로젝트의 한글화 작업이 분기 조건이 아니라 프롬프트 "내용"만 번역했다는 것을 소스코드로 직접 확인할 수 있다.
- ☐ action 리스트의 마지막 항목이 항상 [0.0, 0.0, 0.0, 0.0]이어야 하는 이유(정지 보장)와 그것이 강제되지 않는다는 점을 함께 설명할 수 있다.
- ☐ 웨이크업 중 실행 중이던 이동 명령을 즉시 중단시키는 interrupt 메커니즘을 설명할 수 있다.

더 알아두면 좋은 것

- 이 코드는 eval()로 LLM 응답 문자열을 직접 파싱한다(eval(self.llm_result[...])) — LLM이 요청한 JSON 형식대로 정확히 답하지 않으면 이 부분에서 예외가 날 수 있다. 실무에서는 임의 코드 실행 위험 때문에 eval 대신 json.loads를 쓰는 것이 더 안전하다.
- 이 프롬프트는 "간결하고(5~10자) 재치 있는" 응답 문구를 매번 새로 생성하도록 요청한다 — 같은 명령이라도 매번 다른 음성 응답이 나올 수 있는 것은 버그가 아니라 의도된 프롬프트 설계다.

막혔다면 (더 보기)

🔧 명령했는데 로봇이 안 움직이고 응답만 함

원인: LLM이 "동작을 찾을 수 없다"고 판단해 action을 빈 리스트로 반환함

해결: 프롬프트 예시와 비슷한 표현으로 재시도

🔧 동작이 끝났는데도 로봇이 계속 움직임

원인: LLM이 마지막에 [0,0,0,0]을 안 붙였을 가능성(프롬프트 요청일 뿐 강제 아님)

해결: 수동으로 정지 명령 발행, 필요시 프롬프트 예시를 더 명확히 보강

🔧 eval 관련 에러가 로그에 남음

원인: LLM 응답이 요청한 JSON 형식과 다르게 옴

해결: 프롬프트를 더 구체화하거나 같은 명령을 다시 시도

확인 퀴즈

action 리스트 각 항목의 4개 숫자는 각각 무엇을 의미하는가?

ASR_LANGUAGE가 'Korean'으로 설정되어 있다면 이 코드는 어느 프롬프트를 쓰는가?

interrupt는 언제, 어떻게 실행 중인 동작을 멈추는가?

도전 과제. 이 프롬프트에 "속도는 항상 0.5 이하로 제한"이라는 안전 규칙을 한 줄 추가해보고, LLM이 실제로 그 제한을 지키는 응답을 여러 번 생성하는지 테스트해서 일관성을 확인해보세요.

🎓 프롬프트 엔지니어링과 eval()의 보안 문제

이 프롬프트가 형식 예시(few-shot example)를 2개 넣어둔 것은 우연이 아니다 — LLM에게 규칙을 말로만 설명하는 것보다 실제 입출력 예시를 보여주는 것이 훨씬 안정적으로 원하는 형식을 재현하게 만든다는 것이 프롬프트 엔지니어링의 기본 원칙 중 하나다.

코드가 LLM 응답을 eval()로 파싱하는 것은 보안 관점에서 위험한 패턴이다 — eval()은 문자열을 파이썬 코드로 그대로 실행하기 때문에, 만약 LLM이 (오작동이든 프롬프트 주입이든) JSON이 아닌 임의의 파이썬 코드를 응답에 포함시키면 그대로 실행될 수 있다. 정석적인 방법은 json.loads()로 파싱해서 JSON이 아니면 예외로 처리하는 것이다 — 실행 가능한 코드가 아니라 순수 데이터로만 취급하기 때문에 이런 위험이 없다.

13-1. 쓰레기 분류 로봇 — YOLO 인식부터 잡기·분류까지 하나로 잇기

🕒 1.5시간 (실습 + 결과기록 포함)

선수: 10-1. YOLO 물체 인식

시작 전에. 로봇팔이 실제로 움직이므로 팔 동작 반경 내에 손이나 물건을 두지 않는다.

필요 지식

- waste_classification 노드는 자체 검출 로직이 없다 — 이미 만든 yolo 앱의 /yolo/start·/yolo/stop 서비스를 그대로 호출해서 켜고 끈다. 앱을 새로 만들지 않고 기존 서비스를 조합하는 구조다. (소스: enter_srv_callback → self.start_yolo_client, exit_srv_callback → self.stop_yolo_client)
- WASTE_CLASSES 딕셔너리가 실제 쓰레기 분류 기준이다 — food_waste(BananaPeel·BrokenBones·Ketchup), hazardous_waste(Marker·OralLiquidBottle·StorageBattery), recyclable_waste(PlasticBottle·Toothbrush·Umbrella), residual_waste(Plate·CigaretteEnd·DisposableChopsticks). 이 12개 클래스 이름은 10-1에서 확인한 model_name='garbage' 모델의 클래스 목록과 정확히 일치한다. (소스: WASTE_CLASSES 딕셔너리)
- 카테고리별 목적지 좌표(place_position)가 각각 다르게 하드코딩되어 있다 — 예: residual_waste는 [0.087,-0.225,0.03], food_waste는 [0.025,-0.225,0.03] 등. 실제 분리수거함 배치와 이 좌표가 일치해야 한다. (소스: place_position 클래스 변수)
- 물체가 "잡아도 되는 상태"인지는 연속 프레임 사이 이동 거리(유클리드 거리)로 판단한다 — 거리가 5mm 이내로 10프레임 연속 유지되면(count_still>10) 잡기를 시작하고, 반대로 계속 움직이면(count_move>10) 그 물체를 이번 라운드에서 포기하고 목표 목록에서 제외한다. (소스: main()의 count_still/count_move 로직)
- 실제 팔 자세(관절 각도) 계산은 이 앱이 직접 하지 않는다 — kinematics 서비스(kinematics/set_pose_target)에 목표 좌표(x,y,z)와 자세각(pitch)만 넘기면, 그 서비스가 역기구학(inverse kinematics)을 계산해 서보 값을 돌려준다. "어디로 가야 하는지"는 이 앱이, "어떻게 팔을 굽혀야 닿는지"는 다른 서비스가 맡는 역할 분담 구조다. (소스: pick_and_place.py의 set_pose_target 호출)

단계별로 따라하기

1

enter 호출 → 카메라·YOLO 서브스크립션 활성화 확인

판정기준: 서비스 응답 success

2

set_target으로 목표 카테고리 지정(예: ["recyclable_waste"])

판정기준: target_list 로그 확인

3

enable_transport(true) 호출 → yolo/start가 내부적으로 호출되는지 로그로 확인

판정기준: YOLO 시작 로그 확인

4

목표 카테고리 물체를 카메라 앞에 정지 상태로 놓기 → 잠시 대기 후 잡기 동작이 시작되는지 관찰

판정기준: 잡기 동작 시작

5

잡은 물체가 해당 카테고리의 목적지 좌표로 이송되는지 관찰

판정기준: 올바른 위치에 놓음

6

결과 기록표 작성

판정기준: 4가지 케이스 모두 기록

```
ros2 service call /waste_classification/enter std_srvs/srv/Trigger {}

ros2 service call /waste_classification/set_target \
  interfaces/srv/SetStringList "{data: ['recyclable_waste']}"

ros2 service call /waste_classification/enable_transport \
  std_srvs/srv/SetBool "{data: true}"
```

결과 기록

시도	목표 카테고리	놓은 물체	실제 분류 결과	판정
1	recyclable_waste			성공/실패
2	food_waste			성공/실패
3	목표 리스트에 없는 물체	무시되는지 확인		성공/실패
4	물체를 계속 움직이며 놓기	count_move로 스킵되		성공/실패

한 줄 소감

물체가 안정된 것으로 판단되기까지 체감 대기시간이 얼마나 걸렸는지, 목표 리스트에 없는 물체는 실제로 무시됐는지 서술

완료 체크

- ☐ waste_classification 앱이 자체 검출 로직 없이 yolo 앱의 서비스(/yolo/start, /yolo/stop)를 그대로 호출해 재사용한다는 것을 설명할 수 있다.
- ☐ 물체가 "충분히 멈춰 있다"고 판단하는 기준(연속 10프레임, 유클리드 거리 5mm 이내)을 설명하고 왜 이런 안정성 확인이 필요한지 이해한다.
- ☐ 4가지 쓰레기 카테고리(음식물·유해·재활용·일반)에 속한 실제 클래스 목록을 소스코드에서 확인하고, 각 카테고리가 어느 좌표로 이송되는지 설명할 수 있다.
- ☐ 실제 팔 자세(관절 각도) 계산은 별도의 kinematics 서비스가 담당하고, 이 앱은 "어디로 가야 하는지" 목표 좌표만 넘긴다는 역할 분담 구조를 이해한다.

더 알아두면 좋은 것

- go_home() 함수는 카테고리별로 복귀 대기시간(t)이 다르게 설정되어 있다(recyclable_waste=2.0초, hazardous_waste=1.7초, food_waste=1.4초, residual_waste=1.0초) — place_position까지의 거리·회전량이 카테고리마다 달라서 실측으로 맞춘 값으로 추정되나, 정확한 산출 근거는 소스코드 주석에 남아있지 않아 확인되지 않는다.
- target_list가 비게 되면(목표를 다 처리하면) target_list_temp에서 자동으로 복사해 다시 채운다 — 한 카테고리를 다 치우면 라운드가 자동으로 리셋되어 계속 순찰하듯 동작하는 구조다.

막혔다면 (더 보기)

🔧 물체를 봐도 안 잡음

원인: class_name이 target_list에 없거나 enable_transport가 꺼져있음

해결: set_target으로 정확한 카테고리·클래스명 지정 후 enable_transport(true) 재호출

🔧 물체가 흔들리는 동안 계속 안 잡힘

원인: count_still 기준(연속 10프레임, 5mm 이내)을 못 채움 — 의도된 안정성 확인 동작

해결: 물체를 완전히 고정된 후 대기

🔧 잡았는데 엉뚱한 통에 넣음

원인: WASTE_CLASSES 카테고리 매핑 또는 place_position 좌표가 실제 분리수거함 배치와 안 맞음

해결: place_position 좌표를 실제 배치에 맞게 재조정

확인 퀴즈

waste_classification 앱은 물체 검출을 직접 하는가, 다른 앱의 서비스를 빌려쓰는가?

count_still이 10을 넘어야 잡기를 시작하는 이유는 무엇인가?

로봇팔이 목표 좌표까지 어떻게 굽혀야 하는지는 이 앱이 계산하는가, 다른 서비스가 계산하는가?

도전 과제. WASTE_CLASSES에 새 카테고리(예: 'special_waste')를 하나 추가하고 place_position에도 좌표를 추가해보세요. YOLO 모델 자체는 재학습하지 않으므로, 기존 클래스 중 하나를 새 카테고리에 재배정하는 방식으로 실습해보세요.

14-1. 색상+AprilTag 물체 분류 — 두 가지 인식 방식을 하나로 합치기

⌚ 1.5시간 (실습 + 결과기록 포함)

선수: 13-1. 쓰레기 분류 로봇

시작 전에. 로봇팔이 실제로 움직이므로 팔 동작 반경 내에 손이나 물건을 두지 않는다.

필요 지식

- object_sorting은 YOLO를 쓰지 않는다 — 색상(빨강·초록·파랑) LAB 임계값 검출과 AprilTag(tag36h11 계열, 태그 크기 0.025m) 마커 검출을 각각 수행한 뒤 하나의 목표 리스트로 합친다. (소스: `get_object_pixel_position()`이 색상 blob을, `main()` 안의 `at_detector.detect()`가 AprilTag를 검출해 같은 리스트에 `append`)
- 물체가 안정됐는지 판단하는 기준(유클리드 거리 5mm 이내, 연속 10프레임)은 13-1(쓰레기 분류)과 완전히 동일한 패턴이다 — 다른 앱인데도 같은 안정성 검증 로직을 각자 독립적으로 다시 구현해서 쓰고 있다. (소스: `main()`의 `count_still/count_move` 로직)
- `place_position`은 6개 목적지로 구성된다 — 색상 3종(`green·red·blue`)과 AprilTag 3종(`tag1·tag2·tag3`)이 각각 다른 좌표를 갖는다. (소스: `place_position` 클래스 변수)
- 벤더 소스코드의 `go_home()` 함수에 실제 버그가 있다 — `if self.target[0] in ["bule", "tag1"]:`처럼 "blue"가 "bule"로 오타나 있고, `if/elif` 체인이 매번 `self.target is not None`만 반복 검사하는 구조라 첫 `if`가 한 번 참이 되면 나머지 `elif`는 절대 실행되지 않는다 — 즉 `target`이 `blue·green·red·tag2·tag3` 중 하나면 `t` 값이 아예 계산되지 않는다. 다만 이 함수는 계산된 `t`를 실제로 어디에도 쓰지 않는 죽은 코드라서 프로그램이 멈추거나 에러가 나지는 않는다 — "코드는 있지만 의도한 대로 동작하지는 않는" 실제 사례다. (소스: `go_home()` 147~171행)

선수지식 — 막히면 여기부터

- [동차좌표\(homogeneous coordinates\) 정의와 카메라 투영 연결](#)

글 도입부(동차좌표 정의·수학적 표현)와 후반부 '카메라 projection과의 관계' 부분 위주로 — 중간의 세 부 파생 공식은 건너뛰어도 무방

— gaussian37 블로그 (Computer Vision 전문 블로그, 한국어)

단계별로 따라하기

1

enter → set_target으로 목표(색상 또는 태그) 지정 → enable_sorting(true)

판정기준: 서비스 응답 success

2

색상 물체 하나 놓기 → 분류·이송 관찰

판정기준: 해당 색 목적지로 이송

3

AprilTag 물체 하나 놓기 → 같은 방식으로 분류되는지 관찰

판정기준: 해당 태그 목적지로 이송

4

소스코드에서 go_home()을 직접 찾아 "bule" 오타와 if/elif 구조를 확인 → t 값이 실제로 쓰이는 곳이 있는지 함수 전체를 다시 읽기

판정기준: 버그 지점 특정

5

결과 기록표 작성

판정기준: 3가지 케이스 모두 기록

```
ros2 service call /object_sorting/enter std_srvs/srv/Trigger {}

ros2 service call /object_sorting/set_target \
  interfaces/srv/SetStringBool "{data_str: 'red', data_bool: true}"

ros2 service call /object_sorting/enable_sorting \
  std_srvs/srv/SetBool "{data: true}"
```

결과 기록

시도	목표 종류	놓은 물체	실제 결과	판정
1	색상(예: red)			성공/실패
2	AprilTag(예: tag1)			성공/실패
3	목표로 지정 안 한 색/ 태그	무시되는지 확인		성공/실패
4	go_home() 코드 리뷰 결과	t가 실제로 쓰이는 줄이 ?		찾음/못찾음

한 줄 소감

go_home()의 버그가 실제 로봇 동작에는 왜 영향이 없는지, 소스코드 근거와 함께 자기 말로 설명

완료 체크

- ☐ object_sorting 앱이 색상 인식(LAB blob)과 AprilTag 인식(dt_apriltags) 두 가지 방법을 하나의 목표 리스트로 합쳐 처리한다는 것을 설명할 수 있다.
- ☐ 13-1(쓰레기 분류)과 동일한 안정성 판정(5mm·10프레임) 패턴이 이 앱에서도 독립적으로 재구현되어 있음을 확인한다.
- ☐ 벤더 소스코드에서 실제로 동작하지 않는 코드(go_home())의 오타·도달 불가능한 분기)를 스스로 찾아 내고, 왜 그런데도 프로그램이 멈추지 않는지 설명할 수 있다.
- ☐ place_position이 색상 3종과 AprilTag 3종, 총 6개 목적지로 구성된다는 것을 소스코드에서 확인할 수 있다.

더 알아두면 좋은 것

- AprilTag는 격자무늬 흑백 마커로, 카메라가 코너 4개의 픽셀좌표만 찾으면 마커의 3D 자세(위치+회전)까지 계산할 수 있다 — 색상 인식과 달리 조명 변화에 강하고 회전각도까지 정확히 알 수 있는 것이 장점이다.
- 이번 go_home() 사례처럼, "코드가 존재한다고 해서 그 코드가 실제로 의도한 대로 실행된다는 보장은 없다." 벤더 소스코드를 신뢰하는 것과, 코드의 흐름을 끝까지 따라가며 검증하는 습관은 서로 다른 이야기라는 것을 이번 실습으로 직접 확인했다.

막혔다면 (더 보기)

색상 인식이 안 됨

원인: lab_config.yaml의 색상 임계값이 조명 환경과 안 맞음

해결: 조명을 밝게 하거나 임계값 재조정

AprilTag 인식이 안 됨

원인: 태그 크기(tag_size=0.025m) 또는 family(tag36h11)가 실제 인쇄물과 안 맞음

해결: 정확한 tag36h11 계열 태그를 지정된 크기로 재인쇄

목표로 지정 안 한 물체도 잡음

원인: set_target으로 target_labels가 의도대로 설정되지 않음

해결: set_target 호출 결과를 로그로 재확인

확인 퀴즈

object_sorting은 YOLO를 쓰는가, 다른 방식으로 물체를 찾는가?

go_home()의 elif 체인이 왜 "bule"이 아닌 목표에 대해서는 절대 실행되지 않는가?

이 버그가 실제 로봇 동작에는 문제를 일으키지 않는 이유는 무엇인가?

도전 과제. `go_home()`의 오타("bule"→"blue")와 `elif` 구조를 직접 고쳐서 순수 `if` 3개로 바꿔보세요. 고친 버전과 원본이 실제 로봇 동작에서 다르게 보이는지(`t`가 원래 안 쓰이므로 겉보기엔 차이가 없을 수 있음) 확인 해보고 그 이유를 설명해보세요.

🎓 픽셀 좌표에서 로봇 좌표로 — 카메라 좌표 변환의 원리

카메라가 보는 것은 2D 픽셀 좌표(x, y)뿐이지만, 로봇팔은 3D 실세계 좌표(x, y, z , 단위: m)로 움직여야 한다. 이 변환에는 카메라 내부 파라미터(`intrinsic` — 초점거리·주점 등, `camera_info` 토픽에서 얻음)와 외부 파라미터(`extrinsic` — 카메라가 로봇 기준으로 어디에 어떤 각도로 달려있는지)가 모두 필요하다.

동차좌표(`homogeneous coordinates`)는 3D 좌표에 1을 하나 더 붙여($[x, y, z, 1]$) 회전과 이동(`translation`)을 하나의 행렬 곱셈으로 한 번에 표현하는 방법이다 — 이 소스코드의 `projection_matrix`가 바로 회전(`extrinsic[1]`)과 이동(`extrinsic[0]`)을 하나로 합친 동차변환행렬이다.

이런 계산을 매 프레임 새로 하지 않고 `calibration_*.yaml` 파일에 미리 보정된 값(`offset·scale`)을 저장해두고 재사용하는 이유는, 카메라·로봇팔의 위치 관계가 조립 후에는 거의 바뀌지 않는 고정값이기 때문이다 — 한 번 정밀하게 캘리브레이션해두면 매번 다시 계산할 필요가 없다.

15-1. 손 제스처로 로봇 제어하기 — 궤적의 기울기로 방향을 읽는 법

🕒 1.5시간 (실습 + 결과기록 포함)

선수: 09-1. 로봇팔 첫 동작

시작 전에. 팔이 미리 녹화된 동작을 재생하므로 팔 동작 반경 내에 손이나 물건을 두지 않는다.

필요 지식

- `hand_trajectory_node`는 MediaPipe로 21개 손 랜드마크를 검출하고, 각 손가락의 굽힘 각도 5개를 계산해 `h_gesture()` 함수로 "하나"·"둘"·"다섯"·"주먹" 등 이름 있는 제스처로 분류한다. (소스: `hand_angle()`, `h_gesture()`)
- 상태는 NULL→TRACKING→(발행 후 다시)NULL로 전이한다 — "하나" 제스처가 5프레임 연속 유지 되면 TRACKING 상태로 들어가 검지 손끝(landmark 8) 좌표를 계속 기록하고, "다섯" 제스처가 나오면 지금까지 기록한 궤적을 Points 메시지로 발행하고 다시 NULL로 돌아간다. (소스: `image_proc()`의 State 전이 로직)
- 궤적의 방향 판별은 `hand_gesture_control_node`가 담당한다 — 받은 점들에 `cv2.fitLine`으로 직선을 피팅해 그 기울기 각도를 구하고, 60~90도면 위·아래 스와이프, 0~30도면 좌·우 스와이프로 분류한다. 실제 위/아래·좌/우 구분은 궤적 점들의 좌표 변화 부호(`up_and_down`, `left_and_right`)로 정한다. (소스: `hand_gesture_control_arm()`)
- 이 로봇(MACHINE_TYPE에 'Pro' 포함)은 `hand_gesture_control_arm()` 분기를 탄다 — `ActionGroupController`로 미리 녹화된 동작 시퀀스(action group)를 이름으로 재생한다 ('camera_up', 'hand_control_pick', 'hand_control_place'). 이는 09-1에서 배운 개별 서보 지정, 13-1·14-1에서 배운 좌표 기반 역기구학과는 또 다른 세 번째 로봇팔 제어 방식이다. (소스: `HandGestureControlNode.__init__`의 `if 'Pro' in self.machine_type` 분기)

선수지식 — 막히면 여기부터

• [MediaPipe Hands 21개 손 랜드마크](#)

21개 랜드마크가 정규화 좌표([0,1] 범위)로 표현된다는 정의 부분만 보면 충분
— 푸르고 개발 블로그 (한국어, 원문은 Google MediaPipe 공식문서)

1

hand_trajectory 노드의 /start 호출 → 카메라에 손을 비춰 랜드마크가 그려지는지 확인

판정기준: 손 골격 표시 확인

2

"하나"(검지만 펴기) 제스처를 잠깐 유지 → 궤적 기록(TRACKING) 시작 확인

판정기준: 궤적 선 그려짐

3

손을 위→아래로 움직인 뒤 "다섯"(손바닥 펴기)으로 종료 → 팔이 정해진 동작을 재생하는지 관찰

판정기준: hand_control_pick 재생

4

같은 방식으로 좌→우 궤적 시도 → 다른 동작이 재생되는지 확인

판정기준: hand_control_place 재생

5

결과 기록표 작성

판정기준: 4가지 케이스 모두 기록

```
ros2 service call /hand_trajectory/start std_srvs/srv/Trigger {}

ros2 topic echo /hand_trajectory/points

ros2 service call /hand_trajectory/stop std_srvs/srv/Trigger {}
```

결과 기록

시도	제스처 시퀀스	예상 동작	실제 동작	판정
1	하나→위에서 아래로→다섯			성공/실패
2	하나→왼쪽에서 오른쪽으로→다섯			성공/실패
3	너무 빨리 그은 궤적(점이 적음)			성공/실패
4	"다섯"을 계속 안 보여준 경우	TRACKING이 계속 유지		성공/실패

한 줄 소감

직선 피팅 각도로 방향을 나누는 방식의 한계(대각선으로 그으면 어떻게 되는지 등)를 직접 시도해보고 서술

완료 체크

- ☐ hand_trajectory와 hand_gesture_control 두 노드가 커스텀 토픽(Points)으로 손끝 궤적 데이터를 주고받는 2단계 파이프라인 구조를 설명할 수 있다.
- ☐ "하나"(검지만 펴기) 제스처로 궤적 기록을 시작하고 "다섯"(손바닥 펴기) 제스처로 종료·발행하는 상태전이(state machine) 흐름을 설명할 수 있다.
- ☐ 기록된 궤적에 직선을 피팅(fitLine)해서 그 기울기(각도)만으로 상·하·좌·우 스와이프를 판별하는 원리를 이해한다.
- ☐ 이 프로젝트에서 로봇팔을 움직이는 3가지 방법(개별 서보 지정·좌표 기반 역기구학·미리 녹화된 액션그룹 재생)을 비교해서 설명할 수 있다.

더 알아두면 좋은 것

- 이미 다룬 09-1·13-1·14-1과 비교하면, 이 프로젝트는 로봇팔을 움직이는 서로 다른 세 가지 방식을 모두 갖고 있다 — ① 개별 서보 값을 직접 지정(09-1), ② 목표 좌표만 주면 kinematics 서비스가 역기구학을 계산(13-1·14-1), ③ 미리 녹화한 동작 시퀀스를 이름으로 재생(이번 레슨의 ActionGroupController). 정밀 제어가 필요하다면 ①·②를, 반복되는 정형 동작은 ③을 쓰는 것이 합리적이다.
- 궤적에 점이 하나만 있어도 fitLine은 동작하지만 의미 있는 기울기가 나오지 않는다 — hand_trajectory_node가 이전 점과 5px 미만 차이인 새 점은 기록에서 아예 제외하는 것 (distance(last_point, index_finger_tip) < 5)도 이런 노이즈를 줄이기 위한 장치로 보인다.

막혔다면 (더 보기)

"하나" 제스처를 해도 궤적 기록이 시작 안 됨

원인: 5프레임 연속 유지 조건을 못 채우거나 조명이 어두워 손 검출 실패

해결: 제스처를 더 오래 유지하고 조명을 밝게 조정

궤적은 그렸는데 팔이 반응 안 함

원인: "다섯" 제스처로 정상 종료되지 않아 Points가 발행되지 않음

해결: 손바닥을 확실히 펴서 "다섯"으로 인식시키기

엉뚱한 방향으로 판정됨

원인: 궤적이 대각선에 가까워 60~90도/0~30도 경계에 걸침

해결: 더 수평 또는 수직에 가깝게 크게 그어서 재시도

확인 퀴즈

"하나" 제스처와 "다섯" 제스처는 각각 어떤 역할을 하는가?

궤적의 방향은 어떤 방법으로 판별하는가?

이 로봇에서 로봇팔을 움직이는 세 가지 방식은 각각 무엇인가?

도전 과제. `h_gesture()`의 임계값(`thr_angle=65`, `thr_angle_thumb=53`, `thr_angle_s=49`)을 조금씩 바꿔가며, 자신의 손 크기·카메라 각도에서 제스처 인식이 더 안정적으로 되는 값을 찾아보세요.

16-1. 자율주행 통합 — 차선·신호 등·표지판을 하나의 판단으로 묶기

🕒 2시간 (실습 + 결과기록 포함)

선수: 10-1. YOLO 물체 인식

시작 전에. 로봇이 자율로 주행하므로 트랙 주변 충분한 공간을 확보하고 즉시 정지시킬 수 있도록 대기한다.

필요 지식

- self_driving은 새 인식 로직을 만들지 않는다 — lane_detect.LaneDetector(06-1과 동일한 3-스캔 라인 가중치 0.7·0.2·0.1 방식)로 차선을, yolo 앱의 'traffic' 모델(classes=['go','right','park','red','green','crosswalk'])로 표지판·신호등을 인식한다. (소스: SelfDrivingNode.__init__의 self.lane_detect, self.classes)
- 횡단보도까지의 "거리"는 실제 미터 단위가 아니라 화면에서 횡단보도가 차지하는 y축 픽셀 위치 (crosswalk_distance)로 근사한다 — 이 값이 300을 넘으면(3프레임 연속 확인 후) 감속을 시작하고, 225를 넘으면 주차를 시작하고, 365를 넘으면 우회전 전략을 활성화한다. 전부 실측으로 튜닝된 하드 코딩 임계값이다. (소스: main() 353~410행)
- 빨간불이 감지되어도 무조건 정지하지 않는다 — 신호등 박스의 픽셀 면적이 700을 넘을 때만(화면에 충분히 크게 보일 때만, 즉 충분히 가까울 때만) 정지한다. 너무 멀리서 보이는 빨간불은 오히려 무시하는 것으로, 오탐지·과도하게 이른 정지를 막기 위한 설계로 보인다. (소스: if self.traffic_signs_status.class_name == 'red' and area > 700)
- 우회전 표지가 확인되면, 차선 인식 알고리즘 자체를 속이는 트릭을 쓴다 — 이진화된 차선 이미지 위에 흰색 "가상 선(virtual line)"을 그려 넣어서, 원래 라인트레이싱 로직이 이 가상선을 실제 차선처럼 따라가게 만들어 부드럽게 회전하도록 유도한다. (소스: cv2.line(binary_image, (min_x, y-60), (w, y-60), (255,255,255), 50))

단계별로 따라하기

1

enter → set_running(true)로 자율주행 시작, 노란 선 트랙을 정상 주행하는지 확인

판정기준: 차선 추종 주행

2

빨간불 표지판을 트랙 위 먼 지점에 놓기 → 로봇이 무시하고 계속 가는지 관찰(면적 <700 예상)

판정기준: 정지하지 않음

3

빨간불 표지판을 가까이(로봇 진행 방향 바로 앞) 놓기 → 정지하는지 관찰(면적>700 예상)

판정기준: 정지함

4

횡단보도 표지 + park 표지를 함께 배치 → 감속 후 주차 동작이 실행되는지 관찰

판정기준: 주차 동작 실행

5

우회전 표지를 곡선 구간 앞에 배치 → 자연스럽게 우회전하는지 관찰

판정기준: 우회전 성공

6

결과 기록표 작성

판정기준: 5가지 케이스 모두 기록

```
ros2 service call /self_driving/enter std_srvs/srv/Trigger {}

ros2 service call /self_driving/set_running std_srvs/srv/SetBool "{data: true}"

# 문제 발생 시 즉시 정지
ros2 service call /self_driving/set_running std_srvs/srv/SetBool "{data: false}"
```

결과 기록

시도	상황	예상 동작	실제 동작	판정
1	먼 빨간불	무시하고 주행		성공/실패
2	가까운 빨간불	정지		성공/실패
3	초록불	감속 후 통과		성공/실패
4	횡단보도+park 표지			성공/실패
5	우회전 표지			성공/실패

한 줄 소감

빨간불 정지 거리(면적 700 기준)가 체감상 몇 cm 정도였는지, 우회전이 부드러웠는지 서술

완료 체크

- ☐ self_driving 앱이 06-1(라인트레이싱)의 3-스캔라인 가중치 알고리즘과 10-1(YOLO)의 'traffic' 모델(6개 클래스: go·right·park·red·green·crosswalk)을 그대로 재사용해서 조합한다는 것을 확인할 수 있다.

- ☐ 횡단보도까지의 거리를 y축 픽셀 값으로 근사하고, 여러 하드코딩된 임계값($160 \cdot 225 \cdot 300 \cdot 365$)을 적용해 감속·정지·주차 시점을 판단하는 방식을 설명할 수 있다.
- ☐ 빨간불이 "가까이" 있을 때만(박스 면적 >700) 정지하고 너무 멀면 무시하는 실제 로직을 확인하고 그 이유를 추론할 수 있다.
- ☐ 우회전 표지 감지 시 이진화 이미지에 "가상 유도선"을 그려 넣어 차선 추종 알고리즘이 자연스럽게 회전하도록 유도하는 트릭을 이해한다.

더 알아두면 좋은 것

- 이 lane_detect의 3-스캔라인 가중치($0.7 \cdot 0.2 \cdot 0.1$)는 06-1의 line_following과 동일한 설계 원칙을 공유한다 — 같은 프로젝트 안에서 "라인 추종"이라는 핵심 알고리즘 하나를 여러 앱(line_following, self_driving)이 재사용하고 있음을 확인할 수 있다.
- park_action()은 로봇 조향 방식(Mecanum·Ackermann·기타)마다 완전히 다른 3가지 주차 동작 시퀀스를 갖고 있다 — 메카넘은 옆으로 미끄러져 들어가고, Ackermann은 전후진을 반복하는 3점 주차(3-point turn)에 가까운 동작을 한다.

막혔다면 (더 보기)

빨간불을 봐도 안 멈춤

원인: 신호등 박스 면적이 700 이하 — 아직 너무 멀리 있는 것으로 판단됨

해결: 표지판을 더 가까이 배치하거나 크게 인쇄

횡단보도에서 감속을 아예 안 함

원인: count_crosswalk가 3에 도달하기 전에 표지가 화면에서 사라짐(3프레임 연속 조건)

해결: 표지판이 화면에 안정적으로 계속 보이도록 각도·거리 조정

우회전이 너무 급격하거나 안 됨

원인: 가상선의 y 위치($y-60$)가 트랙 폭·속도와 안 맞음

해결: 심화 과제처럼 관련 상수를 조정해 재시도

확인 퀴즈

self_driving이 차선·표지판을 인식할 때 새로운 인식 알고리즘을 만드는가, 기존 것을 재사용하는가?

빨간불을 봤는데도 로봇이 멈추지 않는 경우는 언제인가?

우회전 표지를 감지했을 때 로봇은 실제로 어떤 트릭으로 회전을 유도하는가?

도전 과제. 빨간불 정지 판단 기준($\text{area} > 700$)을 다른 값으로 바꿔서, 로봇이 더 멀리서 또는 더 가까이서 정지하도록 튜닝해보고 트랙과의 적정 거리를 찾아보세요.

17-1. LLM 함수 호출(function calling) — AI가 도구를 고르고 로봇이 실행한다

🕒 1.5시간 (실습 + 결과기록 포함)

선수: 12-1. LLM 기반 자연어 이동 제어

시작 전에. LLM이 예상 밖의 동작을 생성할 수 있으므로 사방 1.5m 이상 공간을 확보한다.

필요 지식

- 12-1의 프롬프트는 "정해진 JSON 형식으로만 답하라"고 강제하는 방식이었다. 이 앱은 다르다 — tools 리스트에 함수 이름·설명·파라미터 스키마를 등록해두면, LLM이 사용자 요청에 맞는 함수를 스스로 선택해서 "이 함수를 이 인자로 불러줘"라고 응답한다(OpenAI 스타일 function calling). (소스: tools 리스트, 49~365행)
- move_to_location 도구는 destination 파라미터에 enum(["Study_room(서재)", "Bedroom(침실)", "...])을 지정해서, LLM이 절대로 목록에 없는 장소를 만들어내지 못하게 막는다 — 자유 텍스트가 아니라 제한된 선택지로 파라미터를 좁히는 것도 함수 호출 스키마 설계의 일부다. (소스: move_to_location 도구 정의)
- 실제 실행은 동적 디스패치(getattr 등)가 아니라 단순한 문자열 비교 if/elif로 이뤄진다 — LLM이 반환한 tool_name과 'move_to_location', 'line_following' 등을 하나씩 비교해서 맞는 self.메서드()를 호출한다. 새 도구를 추가하려면 tools 리스트와 이 if/elif 양쪽에 각각 추가해야 하는 이중 관리 구조다. (소스: 1021~1051행 부근의 elif tool_name == 'move_to_location': 등)
- robot_move_control 도구의 설명 문구에 영어·한글 불일치가 있다 — 영어 설명은 "it takes 8s to rotate 90°"(90도 회전에 8초)인데, 같은 줄의 한글 번역은 "90° 회전에 걸리는 시간은 4초입니다"라고 되어 있다. 실제 어느 쪽이 맞는지는 로봇에서 직접 재보기 전까지는 확정할 수 없다 — 이런 숫자 불일치는 LLM이 참고하는 설명문에 들어있다는 점에서 더 위험하다(LLM이 잘못된 숫자를 근거로 duration을 계산할 수 있음). (소스: robot_move_control 도구 description, 214행)

선수지식 — 막히면 여기부터

• [LLM function calling\(도구 호출\) 개념](#)

개념 정의와 '문제상황 → 해결책' 도입부 위주 — 뒤쪽 상세 구현 코드는 이 레슨의 실제 소스 발췌로 대체 가능하니 건너뛰어도 됨
— [velog 기술 블로그 \(한국어\)](#)

1

"지금 뭐가 보여?" 같은 질문 → `describe_current_view` 도구가 호출되는지 로그로 확인

판정기준: 해당 도구 호출 로그

2

"빨간 선을 따라가줘" → `line_following(color='red')` 도구가 호출되는지 확인

판정기준: `line_following` 호출

3

"침실로 이동해줘"처럼 존재하는 장소 이동 요청 → `move_to_location` 호출 확인

판정기준: `move_to_location` 호출

4

"은하계로 이동해줘"처럼 존재하지 않는 장소 요청 → `enum`에 없어서 거부/다른 응답이 오는지 관찰

판정기준: 이동 시도 없음

5

결과 기록표 작성

판정기준: 4가지 케이스 모두 기록

```
# tools 스키마 등록 예시 (실제 소스 발췌)
{
  "type": "function",
  "function": {
    "name": "move_to_location",
    "description": "로봇을 지정된 위치로 이동시킵니다",
    "parameters": {
      "type": "object",
      "properties": {
        "destination": {
          "type": "string",
          "enum": ["Study_room(서재)", "Bedroom(침실)", "Kitchen(주방)"]
        }
      }
    },
    "required": ["destination"]
  }
}

# 실행 디스패치 (실제 소스 발췌, if/elif 문자열 비교)
elif tool_name == 'move_to_location':
    destination = arguments['destination']
    res = self.move_to_location(destination)
```

결과 기록

시도	요청 문장	예상 호출 도구	실제 로그	판정
1	화면 설명 요청	describe_current_view		성공/실패
2	라인트레이싱 요청	line_following		성공/실패
3	존재하는 장소 이동	move_to_location		성공/실패
4	존재하지 않는 장소 이동	호출 안 됨 예상		성공/실패

한 줄 소감

12-1 방식과 비교했을 때 이 방식이 더 안정적으로 느껴졌는지, enum 제약이 실제로 잘못된 이동을 막아줬는지 서술

완료 체크

- ☐ 12-1(고정 JSON 형식 강제)과 이 앱의 함수 호출(function calling) 방식의 차이를 설명할 수 있다 — 이번엔 LLM이 "어떤 도구를, 어떤 인자로 부를지"를 스스로 선택한다.
- ☐ tools 리스트에 등록된 12개 이상의 함수(위치 이동·라인트레이싱·물체추적·직접이동 등)가 실제로는 이 노드의 일반 메서드일 뿐이며, 실제 호출은 함수 이름을 문자열로 비교하는 if/elif로 연결된다는 것을 소스코드에서 확인할 수 있다.
- ☐ move_to_location 도구가 enum으로 이동 가능한 장소 목록을 제한해서, LLM이 존재하지 않는 장소로 이동하라고 하는 것을 애초에 막는 설계를 이해한다.
- ☐ 벤더 소스 자체의 영어·한글 설명 불일치(90도 회전 시간 8초 vs 4초)를 스스로 찾아내고, 이런 불일치가 왜 위험한지(LLM이 잘못된 숫자를 참고할 수 있음) 설명할 수 있다.

더 알아두면 좋은 것

- 이 구조는 "LLM이 직접 로봇을 조종하는 코드를 생성"하는 것이 아니라 "이미 검증된 함수 중 하나를 선택해서 호출"하는 것이다 — 12-1에서 다룬 eval()의 보안 위험과 비교하면 훨씬 안전한 설계다. LLM이 아무리 이상한 응답을 해도, tool_name이 등록된 함수 이름과 정확히 일치하지 않으면 아무 동작도 실행되지 않는다.
- get_object_pixel(원래는 object의 오타로 보임)처럼 함수 이름에 오타가 있어도, tools 스키마와 실제 elif 분기 양쪽에 똑같은 오타로 등록만 되어 있으면 시스템은 정상 동작한다 — 함수 "이름"은 프로그래머와 LLM 사이의 임의의 약속일 뿐, 철자가 맞고 틀리고는 동작에 영향이 없다는 것을 보여주는 사례다.

막혔다면 (더보기)

엉뚱한 도구가 호출됨

원인: 요청 문장이 모호해서 LLM이 다른 도구를 선택함

해결: 더 구체적인 문장으로 재요청

🔧 존재하지 않는 장소로 보내달라 했는데 반응이 없음

원인: enum 제약으로 LLM이 애초에 그 값을 선택 못함(의도된 안전장치)

해결: 정상 동작 — get_available_locations로 실제 가능한 장소 확인

🔧 회전 시간이 설명과 다름

원인: 벤더 소스 자체의 영어·한글 설명 불일치(8초 vs 4초)

해결: 실측으로 실제 값을 확인해 기록

확인 퀴즈

12-1의 방식과 이 앱의 function calling 방식은 무엇이 다른가?

move_to_location의 enum은 어떤 문제를 방지하기 위한 것인가?

robot_move_control 설명문의 영어와 한글 중 어느 쪽이 실제로 맞는지 어떻게 확인할 수 있는가?

도전 과제. 실제 로봇으로 $\text{angular.z}=0.5\text{rad/s}$ 로 90도($\pi/2$ 라디안) 회전시켜 걸리는 시간을 스톱워치로 재보고, 소스코드의 영어(8초)·한글(4초) 중 어느 쪽에 더 가까운지 확인해보세요.

18-1. LLM 색상 선택 라인트레이싱 — 세 번째 방식: 함수처럼 생긴 문자열을 정규식으로 읽기

🕒 1시간 (실습 + 결과기록 포함)

선수: 17-1. LLM 함수 호출(function calling)

시작 전에. LLM이 예상 밖의 동작을 생성할 수 있으므로 사방 1.5m 이상 공간을 확보한다.

필요 지식

- 이 노드는 LLM에게 {"action": ["line_following('black')"], "response": "..."} 형식으로 답하라고 요청한다 — action 배열 안의 값이 12-1처럼 순수 숫자 배열도, 17-1처럼 구조화된 tool_name·arguments도 아니라 "함수 호출처럼 생긴 문자열"이다. (소스: PROMPT의 형식 예시)
- 이 문자열을 실행 가능한 코드로 쓰지는 않는다 — 대신 정규식 `r"line_followingW('([^\']+)')"`으로 괄호 안의 색상값만 추출한다(예: `"line_following('red')"` → `"red"`). 문자열 안에 진짜 파이썬 코드가 들어있어도 정규식에 안 맞으면 아무 값도 추출되지 않으므로, 12-1의 `eval()`보다는 안전하지만 17-1의 구조화된 tool calling보다는 더 즉흥적인 방식이다. (소스: `process()`의 `pattern`, `re.search`)
- 색상이 추출되면 06-1에서 배운 그 서비스들(`line_following/set_color`, `line_following/set_running`)을 그대로 호출한다 — 인식 알고리즘도 서비스 인터페이스도 새로 만들지 않고 재사용한다. (소스: `process()`의 `set_target_client`, `start_client` 호출)
- 파일명은 `llm_visual_patrol.py`지만 실제 클래스 이름은 `LLMColorTrack`, ROS2 노드 이름은 `llm_line_following`이다 — 파일명과 실제 기능·이름이 정확히 일치하지 않는 사례로, 코드를 처음 볼 때 파일명만 보고 넘겨짚으면 안 된다는 것을 보여준다. (소스: 파일명 vs class·node 이름)

단계별로 따라하기

1

"검은 선을 따라가줘" 발화 → `line_following` 서비스가 호출되고 실제 추종 주행 시작 확인

판정기준: 라인 추종 시작

2

주행 중 다시 웨이크워드로 불러서 다른 명령 → 진행 중이던 추종이 멈추고 새 명령을 받는지 확인

판정기준: 동작 중단·재대기

3

line_following과 무관한 엉뚱한 요청(예: "노래 불러줘") → action이 빈 배열로 나오고 응답만 오는지 확인

판정기준: 동작 없이 응답만

4

결과 기록표 작성

판정기준: 3가지 케이스 모두 기록

```
# 프롬프트가 요구하는 출력 형식(실제 소스 발체)
{"action": ["line_following('black')"], "response": "알겠습니다"}

# 색상값 추출 (실제 소스 발체)
pattern = r"line_following\('[^']+\)"
match = re.search(pattern, action_string)
if match:
    color = match.group(1) # 예: 'black', 'red'
```

결과 기록

시도	발화 명령	예상 동작	실제 동작	판정
1	검은 선 추종 요청	라인 추종 시작		성공/실패
2	주행 중 재호출로 중단			성공/실패
3	무관한 요청(노래 불러줘 등)	동작 없음		성공/실패

한 줄 소감

12-1·17-1과 비교했을 때 이 방식의 장단점을 자기 말로 정리해서 서술

완료 체크

- ☐ 이 프로젝트가 LLM 출력을 로봇 동작으로 바꾸는 세 번째 방식(문자열로 표현된 가짜 함수 호출을 정규식으로 파싱)을 12-1(숫자 배열)·17-1(구조화된 tool calling)과 비교해서 설명할 수 있다.
- ☐ 06-1의 line_following 서비스(enter·set_color·set_running)가 이번에도 그대로 재사용된다는 것을 확인할 수 있다.
- ☐ 정규식 line_following\('[^']+\)'이 LLM이 만든 "line_following('red')" 같은 문자열에서 실제 색상값만 뽑아내는 원리를 이해한다.
- ☐ 재웨이크업(다시 부르기) 시 진행 중이던 동작이 중단되고 새 명령을 받을 준비 상태로 돌아가는 흐름을 설명할 수 있다.

더 알아두면 좋은 것

- 이제 이 프로젝트 안에서 LLM 출력을 로봇 동작으로 바꾸는 3가지 방식을 전부 확인했다 — ① 12-1의 순수 숫자 배열(가장 단순, eval() 위험 있음) ② 17-1의 구조화된 tool calling(가장 안전·확장하기 좋음, 이중 관리 필요) ③ 이번 레슨의 "함수처럼 생긴 문자열 + 정규식"(그 중간, LLM이 비교적 자유롭게 생성하지만 정규식이 걸러줌). 실무에서 새 LLM 로봇 기능을 설계한다면 ②를 기본으로 삼는 것이 유지 보수에 유리하다.
- 정규식 파싱 방식의 한계 — 이 프롬프트의 "동작 함수 라이브러리"에는 line_following('black') 예시 하나만 적혀 있지만 실제 정규식은 어떤 색상 문자열이든 다 통과시킨다. 즉 프롬프트 문서와 실제 코드가 허용하는 범위가 정확히 일치하지 않는다(문서는 좁게, 코드는 넓게) — 실제 어떤 입력까지 허용되는지는 코드를 직접 읽어야만 확실히 알 수 있다는 교훈을 준다.

막혔다면 (더 보기)

🔧 요청해도 라인트레이싱이 시작 안 됨

원인: LLM이 line_following('색상') 형식과 다르게 응답해 정규식이 매치 안 됨

해결: "~선을 따라가줘" 형태로 더 명확하게 재요청

🔧 색을 지정했는데 다른 색을 따라감

원인: 이전 색상 설정이 초기화되지 않고 남아있음

해결: line_following/enter를 다시 호출해 상태 초기화

🔧 재호출해도 멈추지 않음

원인: wakeup_callback 조건(llm_result 존재 또는 not self.stop)에 안 걸림

해결: 동작이 진행 중일 때 정확히 재호출했는지 타이밍 확인

확인 퀴즈

이 앱이 LLM 출력에서 색상값을 뽑아내는 방법은 무엇인가?

12-1·17-1·이번 레슨의 LLM 출력 처리 방식을 한 문장씩으로 요약해보라.

이 파일의 이름(llm_visual_patrol.py)과 실제 클래스·노드 이름이 다른 것은 무엇을 시사하는가?

도전 과제. 프롬프트의 "동작 함수 라이브러리"에 line_following 외에 다른 동작(예: 존재하지 않는 stop_line_following())을 추가해보고, 실제로 그 함수가 호출되도록 process()의 정규식·처리 로직도 함께 확장해보세요.

19-1. 낙상 감지 로봇 — 사람이 넘어졌는지 픽셀 좌표 평균으로 판단하기

🕒 1시간 (실습 + 결과기록 포함)

선수: 15-1. 손 제스처로 로봇 제어하기

시작 전에. 실습자가 직접 "쓰러지는 연기"를 할 때는 안전한 매트 위에서 진행한다.

필요 지식

- `height_cal()` 함수는 모든 신체 랜드마크의 y좌표(픽셀) 평균을 계산한다 — 사람이 서 있으면 머리부터 발까지 랜드마크가 화면 전체에 고르게 퍼져 평균 y가 중간 정도지만, 쓰러져 누우면 대부분의 랜드마크가 화면 아래쪽에 몰려 평균 y가 커진다. 평균 y가 "이미지 높이 - 120px"보다 크면 낙상으로 의심한다. (소스: `height_cal()`, `image_proc()`의 `if h > image.shape[: -2][0] - 120`)
- 한 번의 판정만으로 결론 내지 않는다 — 연속 3프레임의 판정 결과(1=낙상의심, 0=정상)를 모아 다수(2개 이상)가 낙상을 가리켜야 실제로 경고를 시작한다. 06-1의 3-스캔라인 가중치, 13-1·14-1의 안정성 카운트처럼, 이 프로젝트 전반에 걸쳐 "한 프레임만 보고 판단하지 않는다"는 원칙이 반복해서 나타난다. (소스: `fall_down_count` 리스트, `len==3`일 때 집계)
- 낙상이 확정되면 두 가지 동작이 동시에 별도 스레드로 시작된다 — `buzzer_warn()`은 1000Hz 경고음을 `stop_flag`가 꺼질 때까지 반복하고, `move()`는 로봇을 전진(0.2초)·후진(0.2초)으로 5회 흔든다. 사람이 다시 일어난 것으로 판정되면(`count≤1`) 경고음은 한 번(1900Hz)만 울리고 종료한다 — 같은 `buzzer_warn()` 함수가 `stop_flag` 값에 따라 "계속 울림"과 "한 번만 울림"이라는 정반대 동작을 한다. (소스: `buzzer_warn()`, `move()`)
- 이 로봇(MACHINE_TYPE에 'Pro' 포함)은 시작 시 15-1에서 배운 것과 같은 `ActionGroupController`로 'camera_up' 동작을 재생해 카메라를 위로 향하게 한다 — 몸 전체가 화면에 들어와야 정확한 판정이 가능하기 때문으로 보인다. (소스: `if 'Pro' in self.machine_type: ... self.controller.run_action('camera_up')`)

선수지식 — 막히면 여기부터

- [MediaPipe Pose 신체 랜드마크\(33개 키포인트\) 구조](#)

33개 키포인트가 {x, y, z, visibility}로 표현된다는 정의 부분 위주 — 뒤쪽 스쿼트 판정 응용 사례는 참고만 해도 무방

— velog 기술 블로그 (한국어)

1

노드 실행 → 모니터에 카메라 화면과 신체 랜드마크가 그려지는지 확인

판정기준: 랜드마크 표시

2

정상적으로 서 있는 상태 유지 → 부저·흔들림이 발생하지 않는지 확인

판정기준: 경고 없음

3

매트 위에 눕는 동작(낙상 흉내) → 3프레임 판정 후 부저+흔들림이 시작되는지 관찰

판정기준: 경고 발생

4

다시 일어나기 → 짧은 확인음 한 번과 함께 경고가 종료되는지 관찰

판정기준: 경고 종료

5

결과 기록표 작성

판정기준: 3가지 케이스 모두 기록

```
# 낙상 판정 핵심 로직 (실제 소스 발췌)
def height_cal(landmarks):
    y = [p[1] for p in landmarks]
    return sum(y) / len(y)

# image_proc() 안에서
h = height_cal(landmarks)
if h > image.shape[:2][0] - 120: # 이미지 높이 - 120px
    self.fall_down_count.append(1)
else:
    self.fall_down_count.append(0)

if len(self.fall_down_count) == 3:
    count = sum(self.fall_down_count)
    self.fall_down_count = []
    if count > 1: # 3프레임 중 2번 이상 낙상 판정
        # 부저 경고 + 흔들림 동작 동시 시작
```

결과 기록

시 도	상 황	예 상 반 응	실 제 반 응	판 정
1	정상 서있기	경고 없음		성공/실패
2	눕기(낙상 흉 내)	부저+흔들림 시작		성공/실패
3	다시 일어나 기	확인음 1회 후 종료		성공/실패

한 줄 소감

임계값(120px)이 실제 카메라 설치 높이·각도에서 적절했는지, 오탐지가 있었는지 서술

완료 체크

- ☐ MediaPipe Pose로 얻은 신체 랜드마크의 평균 y좌표 하나만으로 "서 있는지·누워 있는지"를 판단하는 단순한 휴리스틱을 이해한다.
- ☐ 3프레임 다수결(2/3 이상) 판정 방식이 순간적인 오검출을 걸러내는 원리를 설명할 수 있다.
- ☐ 낙상이 감지되면 부저 경고와 로봇의 전후 흔들림 동작이 동시에 실행되는 스텝 구조를 확인할 수 있다.
- ☐ 이 예제는 cv2.imshow로 결과를 표시해서 ROS 토픽으로 스트리밍되지 않는다는 것, 즉 웹 브라우저로는 결과를 볼 수 없고 로봇에 모니터가 연결되어 있어야 한다는 실무적 제약을 이해한다.

더 알아두면 좋은 것

- 이 판정 방식은 카메라 각도·거리에 의존하는 상대적 휴리스틱이다 — 카메라를 낮게 설치하면 서 있는 사람도 화면 아래쪽에 몰려 오탐지가 늘 수 있다. 실제 안전 제품이라면 랜드마크 y좌표 평균보다 더 견고한 특징(예: 신체 세로·가로 비율, 시간에 따른 자세 변화 속도 등)을 함께 쓰는 것이 일반적이다.
- cv2.imshow를 쓰는 예제(이 낙상 감지, qrcode_detector 등)는 로봇에 모니터가 물려있거나 VNC로 데스크톱에 접속했을 때만 결과를 볼 수 있다 — 이 프로젝트의 대다수 앱(line_following, object_tracking 등)처럼 result_image를 ROS 토픽으로 발행해 web_video_server로 스트리밍하도록 바꾸면 브라우저에서도 확인할 수 있게 개선할 수 있다.

막혔다면 (더 보기)

누워도 낙상으로 인식 안 됨

원인: 카메라 각도상 몸 전체가 화면에 안 들어옴

해결: camera_up 액션이 정상 실행됐는지, 카메라 각도·거리 재조정

서 있는데도 계속 오탐지됨

원인: 이미지 높이-120px 기준이 카메라 설치 높이와 안 맞음

해결: 임계값(120)을 실제 환경에 맞게 조정

브라우저로 결과 화면이 안 보임

원인: 이 예제는 cv2.imshow 기반이라 ROS 토픽으로 발행되지 않음(설계상 제약)

해결: 로봇에 직접 모니터 연결 또는 VNC 접속

확 인 퀴즈

낙상 여부를 판단하는 핵심 값은 무엇이며 어떻게 계산되는가?

왜 한 프레임이 아니라 3프레임을 모아서 판정하는가?

이 예제의 결과 화면을 왜 웹 브라우저로 볼 수 없는가?

도전 과제. `image.shape[:-2][0]` - 120의 "120"을 다른 값으로 바꿔가며, 자신의 카메라 설치 각도·높이에서 오탐지가 가장 적은 값을 찾아보세요.

20-1. 사람 따라가기 — 뎁스카메라로 실제 거리를 재는 추적

🕒 1시간 (실습 + 결과기록 포함)

선수: 07-1. 물체 색상 추적

시작 전에. 추적 대상(사람)과 로봇 사이 항상 1.5m 이상 여유를 두고 처음 테스트한다.

필요 지식

- 이 앱은 YOLO('person' 클래스, 10-1에서 다룬 COCO 기본 모델)로 사람의 바운딩박스를 얻고, 박스 중심이 아니라 상단에서 1/3~1/4 지점 내려온 좌표를 추적 기준점(self.center)으로 쓴다 — 사람이 팔다리를 움직여도 상체·머리 근처는 상대적으로 안정적이기 때문으로 보인다. (소스: `get_object_callback()`)
- 거리 측정은 픽셀 위치가 아니라 실제 뎁스카메라 데이터를 쓴다 — 기준점 주변 10×10 픽셀 ROI 안의 깊이값(mm)을 평균 내 cm로 변환한다. 07-1(object_tracking)이 "처음 찾은 y좌표"를 기준 거리로 삼은 것과 달리, 이 앱은 항상 150cm라는 고정 목표거리를 유지하려 한다. (소스: `image_proc()`의 `roi, distances, pid_d.update(distance-150)`)
- 목표거리(150cm) 근처 ± 15 cm, 화면 중앙 근처 ± 30 px는 각각 "그 자리에 있는 것으로 간주"하는 데드존이다 — 값이 미세하게 흔들려도 로봇이 계속 미세 반응하며 떠는 것을 막는 장치다. (소스: `if abs(distance-150)<15: distance=150, if abs(self.center[0]-w/2)<30: self.center[0]=w/2`)
- 이 앱도 Ackermann 조향 로봇에서는 회전각·회전반경 계산으로, 그 외에는 angular.z 직접 지정으로 갈리는 동일한 패턴(03-2·07-1·08-1에서 반복된)을 따른다. (소스: `if 'Acker' not in self.machine_type: ... else: ...`)

단계별로 따라하기

1

노드 실행 → 카메라 앞에 서서 중심점(노란 점)이 몸에 표시되는지 확인

판정기준: 중심점 표시

2

앞뒤로 움직이며 로봇이 150cm 거리를 유지하려 하는지 관찰

판정기준: 거리 유지

3

좌우로 움직이며 로봇이 회전으로 따라오는지 관찰

판정기준: 회전 추적

4

여러 명이 동시에 화면에 있을 때 어떤 사람을 따라가는지 관찰

판정기준: 대상 확인

5

결과 기록표 작성

판정기준: 3가지 케이스 모두 기록

```
# 거리·각도 제어 핵심 로직 (실제 소스 발췌)
roi = self.depth_frame[h_1:h_2, w_1:w_2] # 기준점 주변 10x10 픽셀
distances = roi[np.logical_and(roi > 0, roi < 40000)]
distance = int(np.mean(distances) / 10) # mm -> cm

if abs(distance - 150) < 15:
    distance = 150 # 목표거리(150cm) 근처는 데드존
    self.pid_d.update(distance - 150)
    self.linear_x = -common.set_range(self.pid_d.output, -0.4, 0.4)

if abs(self.center[0] - w/2) < 30:
    self.center[0] = w/2 # 화면 중앙 근처도 데드존
    self.pidAngular.update(self.center[0] - w/2)
```

결과 기록

시도	상황	예상 동작	실제 동작	판정
1	전후 거리 유지	150cm 유지		성공/실패
2	좌우 회전 추적			성공/실패
3	다수 인원 등장	어느 한 명을 따라감		성공/실패

한 줄 소감

07-1과 비교했을 때 이 앱의 추적이 더 안정적으로 느껴졌는지, 다수 인원 상황에서 어떤 사람을 따라갔는지 서술

완료 체크

- ☐ 07-1(픽셀 y좌표로 거리를 추정하는 방식)과 이 앱(덱스카메라로 실제 mm 단위 거리를 재는 방식)의 차이를 비교해서 설명할 수 있다.
- ☐ YOLO의 COCO 기본 모델(10-1에서 배운 80개 클래스)에서 'person' 클래스 하나만 추적 대상으로 삼는 것을 확인할 수 있다.
- ☐ 목표 지점 주변 작은 사각형(ROI) 안의 깊이값 평균을 구하고, 목표거리(150cm) 근처는 그대로 유지하는 데드존(dead zone) 설계를 이해한다.

- ☐ 사람의 바운딩박스에서 "중심점"이 아니라 상단에서 일정 비율 내려온 지점을 추적 기준으로 삼는 이유를 추론할 수 있다.

더 알아두면 좋은 것

- `get_object_callback()`은 여러 사람이 감지되면 마지막으로 처리된 `person` 객체의 좌표로 `self.center`를 계속 덮어쓴다 — 반복문에 `break`가 없어서 "가장 먼저 발견한 사람"이 아니라 "목록에서 가장 나중에 나온 사람"을 따라가게 될 수 있다. 여러 명이 있는 환경에서는 예측하지 못한 대상을 따라갈 수 있다는 뜻이다.
- 07-1의 픽셀 기반 거리 추정과 이 앱의 뎁스카메라 기반 거리 측정을 비교하면, 뎁스카메라 방식이 조명·물체 크기에 덜 민감한 대신 유효 거리 범위(40000mm 이내)와 뎁스카메라 자체의 오차 범위에 의존한다는 트레이드오프가 있다.

막혔다면 (더 보기)

사람을 따라가지 않음

원인: YOLO가 'person'으로 인식 못함(조명·자세 문제) 또는 뎁스값이 유효범위 밖

해결: 밝은 곳에서 정면으로 서서 재시도

거리는 맞는데 회전이 안 됨

원인: `center[0]`이 화면 중앙 데드존($\pm 30\text{px}$) 안에 계속 있음

해결: 더 크게 좌우로 이동해서 재시도

엉뚱한 사람을 따라감

원인: 여러 명이 감지될 때 마지막 처리된 사람이 채택되는 구조

해결: 심화 개념 참고 — 의도된 동작이 아니라 코드 구조상 나타나는 현상

확인 퀴즈

이 앱이 사람과의 거리를 재는 방법은 07-1과 어떻게 다른가?

데드존은 왜 필요한가?

여러 사람이 동시에 화면에 있으면 로봇은 누구를 따라가는가?

도전 과제. `get_object_callback()`을 고쳐서 "가장 가까운 사람"이나 "가장 큰 바운딩박스를 가진 사람"을 따라가도록 로직을 바꿔보고 어떻게 달라지는지 실험해보세요.

🎓 데드존(dead zone) 설계 — 왜 목표값 근처에서는 일부러 반응하지 않는가

센서 값은 완벽하게 고정되지 않는다 — 사람이 가만히 서 있어도 뎀스카메라가 재는 거리(distance)나 바운딩박스 중심(center)은 프레임마다 몇 mm·몇 px씩 미세하게 흔들린다. 만약 컨트롤러가 이 미세한 흔들림에도 매번 반응한다면, 로봇은 목표에 도달한 뒤에도 계속 미세하게 전진·후진·회전을 반복하며 떠는 현상(hunting, 헌팅)이 생긴다 — 에너지 낭비는 물론 기구부 마모, 불안정해 보이는 동작으로 이어진다.

이 코드는 목표값 주변에 **허용 구간(tolerance band)**을 두어 이 문제를 막는다 — if $\text{abs}(\text{distance}-150)<15$: $\text{distance}=150$ 은 실제 거리가 135~165cm 사이 어디든 강제로 150(=오차 0)으로 취급해버린다. PID 컨트롤러 입장에서는 오차가 정확히 0이니 아무 것도 하지 않는다. 화면 중앙 판정($\text{abs}(\text{center}[0]-w/2)<30$)도 동일한 원리다.

데드존 폭은 트레이드오프다 — 너무 좁으면(예: $\pm 2\text{cm}$) 여전히 자잘한 떨림이 남고, 너무 넓으면(예: $\pm 50\text{cm}$) 사람이 실제로 30cm 가까워져도 로봇이 반응하지 않아 "둔하다"고 느껴진다. 이 코드가 택한 $\pm 15\text{cm}$ · $\pm 30\text{px}$ 이라는 구체적인 수치는 실측으로 튜닝된 값으로 보이며, 정답이 하나로 정해진 공식은 없고 "실제 하드웨어의 노이즈 크기"에 맞춰 결정해야 하는 설계 파라미터다. 가정용 보일러 온도조절기가 목표온도 ± 0.5 도 안에서는 켜고 끄기를 반복하지 않는 것도 같은 원리(히스테리시스, hysteresis)다.

21-1. 지정한 사람만 따라 움직이기 — 몸짓 인식 + 옷 색상으로 대상 고정하기

🕒 1.5시간 (실습 + 결과기록 포함)

선수: 20-1. 사람 따라가기

시작 전에. 로봇이 몸짓에 따라 움직이므로 사방 1.5m 이상 공간을 확보한다.

필요 지식

- "허리에 손 얹기" 자세(양팔 굽힘 각도가 특정 범위)가 5프레임 연속 감지되면 캘리브레이션 모드로 들어간다 — 이 자세 자체는 이동 명령이 아니라 "지금부터 이 사람을 기준으로 삼겠다"는 신호다. (소스: `image_proc()`의 `angle_list` 조건, `count_akimbo`)
- 캘리브레이션 중에는 06-1·07-1에서 쓴 것과 같은 `ColorPicker` 클래스를 재사용하되, 이번엔 라인이나 물체가 아니라 신체 중심점(양쪽 어깨·엉덩이 평균)에 적용해 그 사람 옷의 색상을 읽어 저장한다 (`lock_color`). 이후 매 프레임 같은 지점의 색을 다시 읽어 저장된 색과의 차이(`get_dif`, RGB 채널 차이 합)가 50 미만이면 "같은 사람"으로 판단해 제어를 허용한다 — 얼굴 인식이 아니라 옷 색상으로 특정 인물을 구분·추적하는 실용적 트릭이다. (소스: `color_picker`, `get_dif()`, `can_control`)
- 팔·다리를 들었는지는 관절 사이 거리의 제곱 비율로 판별한다 — 예: $(\text{엉덩이-어깨 거리}^2) / (\text{엉덩이-손목 거리}^2)$ 가 1 미만이면 팔을 든 것으로 본다. 4프레임을 모아 3프레임 이상 감지되면(`count>2`) 이동을 시작하고, 2프레임 이하로 떨어지면(`count≤2`) 정지한다 — 06-1·13-1·19-1과 같은 "여러 프레임을 모아 판단" 원칙이 다시 나타난다. (소스: `joint_distance()`, `move_status` 판정)
- Ackermann 조향 로봇에서는 팔을 들었을 때 `Twist(cmd_vel)`를 쓰지 않는다 — 대신 조향 서보 각도를 직접 지정하고, 좌우 바퀴 모터(id 2, 4)에 각각 다른 회전수(`rps`)를 직접 명령해 8초간 회전한 뒤 정지시키는 저수준 제어를 쓴다. 이 커리큘럼의 다른 모든 레슨은 `/controller/cmd_vel` 토픽(`Twist`)만 썼는데, 이 앱은 그 위에 있는 하드웨어 제어 계층(`MotorsState`)까지 직접 건드리는 유일한 사례다. (소스: `move()` 안의 `MotorState`·`MotorsState` 발행)

단계 별로 따라 하기

1

노드 실행 → 카메라 앞에서 허리에 손 얹기 자세 유지 → 캘리브레이션 완료 알림(부저) 확인

판정기준: 부저 알림

2

다른 색 옷을 입은 사람이 끼어들었을 때 제어가 안 되는지 확인
(`can_control=False` 예상)

판정기준: 제어 불가 확인

3

캘리브레이션된 사람이 한쪽 팔을 들어올리기 → 로봇이 해당 방향으로 이동하는지 관찰

판정기준: 이동 반응

4

팔을 내리기 → 로봇이 정지하는지 확인

판정기준: 정지

5

결과 기록표 작성

판정기준: 4가지 케이스 모두 기록

```
# 색상 캘리브레이션 + 인물 판정 핵심 로직 (실제 소스 발췌)
if self.lock_color is None and self.current_color is not None and self.calibrating:
    self.lock_color = self.current_color[1] # 옷 색상 저장
    self.buzzer_warn() # 캘리브레이션 완료 알림

if self.lock_color is not None and self.current_color is not None:
    res = get_dif(list(self.lock_color), list(self.current_color[1]))
    self.can_control = (res < 50) # 색상 차이가 작으면 같은 사람으로 판단
```

결과 기록

시 도	자 세 · 상 황	예 상 동 작	실 제 동 작	판 정
1	캘리브레이션	부저 알림		성공/실패
2	다른 색 옷 사람 개입	제어 안 됨		성공/실패
3	원팔 들기			성공/실패
4	팔 내리기	정지		성공/실패

한 줄 소감

옷 색상 기반 인물 구분이 실제로 얼마나 안정적이었는지, Ackermann 로봇이라면 회전 동작이 다른 레슨과 어떻게 다르게 느껴졌는지 서술

완료 체크

- ☐ "허리에 손 얹기" 자세가 이동 명령이 아니라 "지금부터 이 사람의 옷 색을 기준으로 삼겠다"는 캘리브레이션 트리거로 쓰인다는 것을 이해한다.
- ☐ 06-1·07-1에서 이미 배운 ColorPicker가 이번엔 라인이나 물체가 아니라 "사람 몸통 중심점"에 적용되어, 신체 인식과 색상 인식을 결합해 특정 인물을 구분하는 원리를 설명할 수 있다.
- ☐ 팔·다리를 들어올리는 자세가 관절 사이 거리 비율로 판별되고, 4프레임 다수결로 확정되는 과정을 설명할 수 있다.
- ☐ Ackermann 조향 로봇의 회전 동작이 이 앱에서는 Twist(cmd_vel)가 아니라 개별 바퀴 모터에 직접 회전수(RPS)를 지정하는 저수준 제어로 구현되어 있다는 것을 소스코드에서 확인할 수 있다.

더 알아두면 좋은 것

- 옷 색상 기반 인물 구분은 값싸고 계산이 가벼운 대신, 비슷한 색 옷을 입은 다른 사람이 나타나면 쉽게 혼동될 수 있는 한계가 있다(get_dif 임계값 50이 상당히 관대함). 실제 사람 재식별(re-identification)에는 이보다 정교한 특징(옷 무늬·얼굴·걸음걸이 등)을 함께 쓰는 것이 일반적이다.
- Twist(cmd_vel) 대신 MotorsState로 개별 바퀴를 직접 제어하는 것은 더 정밀한 제어가 가능하지만, cmd_vel 위에서 동작하는 다른 안전장치(반복 발행 관례 등)를 우회한다는 뜻이기도 하다 — 왜 이 특정 동작만 더 낮은 계층으로 내려갔는지는 소스 주석에 명시되어 있지 않다.

막혔다면 (더 보기)

캘리브레이션이 안 됨

원인: 허리에 손 얹기 자세가 각도 조건과 안 맞음

해결: 카메라를 정면으로 보고 자세를 더 명확하게 취하기

캘리브레이션했는데 제어가 안 됨

원인: 옷 색상이 배경·조명과 비슷해 get_dif 값이 임계값(50)을 넘음

해결: 더 뚜렷한 색 옷으로 재시도

팔을 들어도 반응이 늦음

원인: 4프레임 다수결 조건(3/4 이상)을 못 채움

해결: 자세를 좀 더 오래 유지

확인 퀴즈

"허리에 손 얹기" 자세는 이동 명령인가, 다른 용도인가?

이 앱이 특정 인물을 구분하는 방법은 무엇인가?

Ackermann 로봇의 팔 동작 반응이 다른 레슨들과 다른 점은 무엇인가?

도전 과제. `get_dif()`의 판정 임계값(50)을 낮추거나 높여보고, 비슷한 색 옷을 입은 다른 사람과의 혼동이 얼마나 줄거나 느는지 실험해보세요.